

Instruction Granularity as Planning Width: Formalizing and Benchmarking for Vision-Language-Action Models

Author Name

Affiliation

email@example.com

Abstract

Real-world users provide instructions at varying levels of granularity, yet this remains an uncontrolled and unmeasured variable in Vision-Language-Action (VLA) research, leaving its impact unknown. We formalize granularity as the *planning demand* required to ground language into actions, and adapt *rule width* from width-based planning to quantify this demand. To isolate and evaluate this variable, we introduce Mini-BEHAVIOR-Gran, extending all 20 complex tasks with: (i) a rule-set bank providing instruction variants across up to six granularity levels, and (ii) a dynamic instructor that delivers valid instructions at each step. Our analysis reveals a non-monotonic, U-shaped relationship between granularity and performance, with peaks at fine and coarse extremes. We demonstrate an asymmetry in generalization: models trained on coarse instructions transfer to fine ones, but not vice versa. Furthermore, we diagnose how coarse instructions can induce language-ignoring shortcuts due to high visual predictability, and identify mixed-granularity training regimens that best balance task performance with language grounding. Our findings establish instruction granularity as a fundamental dimension for diagnosing and improving multimodal alignment in VLAs.

1 Introduction

When deploying Vision-Language-Action (VLA) models in the real world, a key challenge arises from natural variation in instruction granularity. Users may provide anything from highly precise commands to vague intentions (Clark, 1996). Handling instruction granularity is essential for human-agent interaction (Norman, 2002), yet current research faces two interrelated gaps that prevent a systematic answer.

First, at the conceptual levels, the community has not yet reached a consensus on how to formally define or measure instruction granularity. A prevalent practice in hierarchical LLM agents (R. Yang, F. Zhang, and Hou, 2024) and recent VLA work (Myers et al., 2024; Shi et al., 2025; Zhong et al., 2025) is to operationalize granularity as the *level of task decomposition*, i.e., whether an instruction specifies only the fi-

nal goal or also enumerates subgoals. While intuitive, *decomposition depth* is not a stable notion of granularity but dependent on the task structure: some tasks are intrinsically difficult yet do not admit meaningful hierarchical breakdowns due to *interleaved subgoals* (Sussman, 1975), whereas even simple tasks can be divided into many steps. In parallel, existing Natural Language Processing (NLP) studies of “granularity” typically focus on linguistic aspects, such as the *level of detail* or *discourse structure* of text (Dong and Lapata, 2018; H. Huang et al., 2025), rather than adopting an embodied-agent-centric view to measure an instruction’s *informativeness* in terms of reducing the effort required to generate action plans. We instead depart from these proxies by defining instruction granularity as the *planning demand* imposed by an instruction, i.e., how much internal search an agent must perform to ground the instruction into a concrete, executable plan. This framing explicitly decouples granularity from dedicated choices of task decomposition and yields a directly quantifiable measure within *classical planning* frameworks, which is a better approach for analyzing and benchmarking VLA models.

Second, at the empirical level, variations of language granularity in existing VLA benchmarks is entangled with other factors. For example, benchmarks such as BEHAVIOR-1K, which focus on long-horizon activities, typically use abstract goal descriptions, whereas suites like LIBERO, which emphasize short-horizon robotic manipulation, adopt highly concrete commands (C. Li, R. Zhang, et al., 2022; X. Li et al., 2024; Liu et al., 2023; Mu et al., 2025; Shridhar et al., 2020a). Therefore, it is difficult to disentangle a model’s failure from either challenging perception, or complex low-level control as opposed to the impact of instruction granularity. In short, the field lacks a benchmark that holds tasks fixed while providing parallel instruction sets with rigorously controlled instruction granularity. To this end, the contributions of our paper are:

(1) A Domain-level Formalization: We formalize instruction granularity as *rule width*, a concept adapted from *width-based planning* (Bonet, Francès, and Geffner, 2019; Drexler, Seipp, and Geffner, 2024). Intuitively, it measures how many state variables (i.e., *features*) of the world state must be maintained **concurrently** when fulfilling an instruction (§ 3). This metric enables consistent comparison across tasks within the same domain, as it is defined over the shared domain-level feature set Φ . More importantly, instruction granularity now

reflects the *informativeness* of an instruction when an embodied agent attempts to ground it into an action plan.

(2) A Granularity-Controlled Benchmark: We introduce MINI-BEHAVIOR-GRAN, where each task is paired with multiple instruction variants spanning different granularity levels (§ 4). It is the first-of-kind testbed that isolates instruction granularity as an independent variable, so as to enable controlled studies of its effects on VLA agents.

(3) A Granularity-Centric View of Language Grounding: We reveal a non-monotonic, U-shaped relationship between instruction granularity and VLA performance, identifying productive learning regimes at the extremes of Fine and Coarse instructions (§ 5). While prior work attributes “language-ignoring” primarily to statistical biases in training data, we provide a complementary granularity-centric perspective. We argue that instruction granularity is a structural driver of the trade-off between task success and language reliance. Through extensive evaluation across three action-decoding archetypes (Autoregressive, Diffusion, and Parallel), we demonstrate that coarse instructions can incentivize a bypass of linguistic grounding due to their higher predictability from visual state alone, whereas mixed-granularity training, particularly our proposed *Axial-F* setting, best balances overall performance and language grounding.

Instruction granularity shapes the learning dynamics of VLA models trained on multimodal datasets. It introduces a dual risk: overly fine-grained instructions may degrade language into a mere identifier for specific state-action sequences, encouraging memorization rather than generalized understanding; conversely, overly coarse instructions can weaken language-action alignment, encouraging the model to exploit shortcuts in other data modalities—a risk that has been difficult to isolate and measure. Through our formalization and controlled benchmark, we provide the first systematic study of instruction granularity in VLAs, establishing it as a measurable dimension for future research on language grounding and multimodal alignment.

2 Related Work

Instruction Abstraction and Granularity. In this work, *Instruction granularity* refers to the extent to which state and action constraints are explicitly specified versus left for the agent to infer when grounding language into actions. This is distinct from, yet often conflated with the concept of *linguistic abstraction*. The term *abstraction* is viewed as *representational compression*—using a compact description to stand for multiple instances or executions. This definition is common in prior work of computational thinking (Lachmy et al., 2022; Wing, 2010; Zheng et al., 2024). Importantly, granularity can vary independently of abstraction: “tidy the room” is coarse but non-abstract (it leaves the exact required actions unspecified, yet it lacks representational compression.), whereas “repeat wiping until clean” is relatively specific yet abstract (it prescribes an action but uses a loop condition to compress multiple executions).

While not framed as the term “granularity”, we acknowledge that a rich body of work has focused on augmenting language specificity and its impact, for example through coarse-

State Progression	Concurrent Features Tracked (Sequence of Tuples)	Width
Initial State	$\{\langle \rangle\}$	0
Navigate to rag	$\{\langle p_r = X \rangle, \dots, \langle p_r = 0 \rangle\}$	1
Pick up rag	$\{\langle p_r = 0 \rangle, \langle H_r \rangle\}$	1
Navigate to sink	$\{\langle H_r, p_s = X \rangle, \dots, \langle H_r, p_s = 0 \rangle\}$	2
Turn on sink	$\{\langle H_r, p_s = 0 \rangle, \langle T_s \rangle\}$	2
Soak rag	$\{\langle H_r, T_s, p_s = 0 \rangle, \langle S_r \rangle\}$	3
Navigate to Shoe 1	$\{\langle H_r, S_r, p_{f1} = X \rangle, \dots, \langle H_r, S_r, p_{f1} = 0 \rangle\}$	3
Clean Shoe 1	$\{\langle H_r, S_r, p_{f1} = 0 \rangle, \langle C_1 \rangle\}$	3
Navigate to Shoe 2	$\{\langle H_r, S_r, C_1, p_{f2} = X \rangle, \dots\}$	4
Clean Shoe 2	$\{\langle H_r, S_r, C_1, p_{f2} = 0 \rangle, \langle C_1, C_2 \rangle\}$	4
Put rag on floor	$\{\langle C_1, C_2, H_r \rangle, \dots, \langle C_1, C_2, on_{floor_r} \rangle\}$	3

Table 1: State progression and width for the “Cleaning k Shoes” ($k = 2$) w/o instructions. Width scales as $w = k + 2$.

to-fine semantic parsing (Dong and Lapata, 2018; Z. Li et al., 2020). Other work generates compositional instructions via constraint dropout and sub-task recombination (H. Huang et al., 2025), or construct counterfactual instructions to augment data diversity (Glossop et al., 2025).

In embodied instruction following, varying instruction detail is often handled via a simple two-part decomposition: high-level plans are converted into sequences of low-level steps before being grounded into actions (Shi et al., 2025; R. Yang, F. Zhang, and Hou, 2024). Most recent VLA works, however, operate with static language instructions (W. Li et al., 2025; Ma et al., 2025). Some models like InstructVLA and ThinkAct explore having chain-of-thought reasoning steps before action decoding (C.-P. Huang et al., 2025; S. Yang et al., 2025). Despite this, quantitatively controlled studies on multi-level instruction granularity remain scarce in embodied AI domain.

Language Sensitivity vs. Ignoring in VLA. Recent studies present two seemingly contradictory findings. On one hand, VLA models appear sensitive to language variations, where semantically equivalent rephrasing of seen instructions leads to performance drops (Akula et al., 2022; Karnik et al., 2025); this suggests a reliance on superficial word patterns. On the other hand, recent analyses identify a “language-ignoring” behavior, where models over-rely on visual context and bypass instructions entirely (Fei et al., 2025; Xing et al., 2025; Xu et al., 2025). We provide a comprehensive granularity-centric view that reconciles these findings: instruction granularity modulates the balance between language sensitivity and ignoring, with overly coarse instructions leading to language-ignoring shortcuts.

3 Rule Width as Instruction Granularity

3.1 Running Example

We first illustrate *planning width* computation using a running example. Table 1 shows the desired state progressions of a task where two shoes are cleaned using a rag, and the rag is placed on the floor afterwards. The key **environment dynamics** are: (1) the rag must be soaked before it can clean shoes, and (2) the sink must be turned on to soak the rag. We represent states using a set of task-relevant feature variables (Φ): (i) boolean features H_r (holding rag), T_s (sink is turned on), S_r (rag is soaked), C_i (shoe i is cleaned), on_{floor_r} (rag is on the floor); and (ii) spatial features p_r, p_s, p_f (distance to the rag, sink, and nearest uncleaned shoe, respectively).

First, approaching the rag is a navigation-only segment: the desired state progression is to decrease the distance feature p_r until $p_r = 0$. Thus, the width is 1 as only a single feature needs to be tracked. In this environment dynamics, $p_r = 0$ enables the state transition to H_r . Thus, the rag can be obtained at $p_r = 0$ and the width remains 1. Subsequently, H_r must be maintained while progressing to next states, resulting in a width of 2. The rag becoming soaked requires simultaneous maintenance of three features, i.e., at the sink ($p_s = 0$), holding the rag (H_r), and having the sink turned on (T_s), which yields a width of 3. After soaking, C_1 can be obtained by tracking H_r , S_r , and p_f , giving width 3. The subtlety arises when cleaning the second shoe: one has to track the same features as before, plus maintaining the cleanliness of the first shoe (C_1), which increases the width to 4.

Why no action model? Agents’ action sets constrain how state transitions are executed, but *width* focuses on state-level feature dependencies induced by the environment dynamics. Different agents (e.g., a rover vs. a drone) may have different action spaces to achieve the same feature change, so emphasizing state progressions promotes generalization of width-based concepts across different embodiments.

The impact of *instruction granularity* on VLA behavior remains underexplored largely because the field lacks a **formal, quantifiable, cross-task** measure of granularity. The most common heuristic, *decomposition depth* (Myers et al., 2024; Shi et al., 2025), is unstable as discussed in § 1. To bridge this gap, we formalize *instruction granularity* as *planning demand*—the internal search required to ground an instruction into an executable plan. This perspective captures the intuitive difference between fine- and coarse-grained instructions: fine-grained instructions provide a “breadcrumb trail” that narrows the search, whereas coarse instructions leave a much larger solution space to be explored to reach a state that fulfills the instruction.

We operationalize planning demand using **rule width** (w), adapting the notion of *planning width* from classical planning (Lipovetzky and Geffner, 2012) and its extension to *policy sketches* (Bonet and Geffner, 2021; Drexler, Seipp, and Geffner, 2022, 2024). We first introduce the underlying task representation, then model instructions as sets of “if-then” rules that decompose tasks, and finally define rule width as a measure of instruction granularity for these rule sets.

3.2 Symbolic Task Representation

While VLAs operate on raw multimodal observations, we adopt a symbolic, relational state abstraction. Many embodied benchmarks provide such structured representations—object properties and relations—that can be expressed as *fluents* to form symbolic state or goal conditions (X. Li et al., 2024; Shridhar et al., 2020b). Formally, we consider a classical planning problem $P = \langle \mathcal{F}, \mathcal{A}, I, G \rangle$, where $\mathcal{F}$ is a set of fluents, $\mathcal{A}$ a set of actions, I the initial state, and G the goal.

To bridge raw states and high-level instructions, we define a set of **features** $\Phi = \{\phi_1, \dots, \phi_n\}$, where ϕ_i is a Boolean or integer-valued function over state space S . These features serve as domain-level abstractions that capture task-relevant aspects of the environment. Thus, each s is associated with a **feature valuation vector** $\phi(s) = (\phi_1(s), \dots, \phi_n(s))$.

3.3 Instructions as Rule Sets $\mathcal{R}$

To study instruction granularity in a controlled manner, we formalize each instruction a symbolic interpretation as a set of “if-then” rules $\mathcal{R} = \{r_1, \dots, r_m\}$ defined over the feature space Φ . This rule-based representation does not aim to capture the full expressiveness of natural language; instead, it distills the constraint/goal structure that governs the computational cost incurred by a symbolic search algorithm (i.e., planning demand). Specifically, each r_i is of the form $C_{r_i} \mapsto E_{r_i}$:

- C_{r_i} (the *condition*) is a conjunction of feature constraints. For a Boolean feature p , a constraint is p or $\neg p$; for a numerical feature n , it is $n = 0$ or $n > 0$.
- E_{r_i} (the *effect*) is a conjunction of feature changes. For a Boolean feature p , an effect is p , $\neg p$, or $p?$ (allowing any change); for a numerical feature n , it is $n \downarrow$ (decrease), $n \uparrow$ (increase), or $n?$ (any change).

These forms capture the core semantics of the instructions we study. A rule r_i is **fulfilled** by a state sequence $(s, \dots, s')$ if $\phi(s) \models C_i$ and the pair $(\phi(s), \phi(s'))$ satisfies the changes in E_i (e.g., $\forall (n \downarrow) \in E_{r_i}, \phi_n(s') < \phi_n(s)$) while keeping unmentioned features constant (i.e., $\phi_j(s') = \phi_j(s)$ for $j \notin E_i$).

In our experiments, all natural-language instructions presented to the VLA agent are designed such that their symbolic interpretations, i.e., the rule sets $\mathcal{R}$ over features Φ , are **well-formed** (i.e., goal-separating and terminating) and it is guaranteed that at least one **sound** and **closed** (Bonet and Geffner, 2021) policy exists on all benchmark instances, so rule-following ensures dead-end-free reachability. This rule-based framing explicitly exposes the latent subgoal structure, enabling controlled variation of how much structural guidance the instruction provides.

3.4 Rule Width as a Granularity Measure

We quantify instruction granularity through *rule width* (w), a metric that captures the peak concurrent planning demand required to ground language into action. The intuition is rooted in the mechanism of **Iterated Width (IW)** search (Lipovetzky and Geffner, 2012). $IW(k)$ is a novelty-guided breadth-first search algorithm that retains a state only if it makes some previously unseen combination of k feature values true. Thus, if a rule r_i has width $w(r_i) = 2$, then reaching its effect E_i from a state s with $\phi(s) \models C_i$ can be fulfilled by tracking at most two features concurrently (to be made precise with a running example in § 3.5); consequently, $IW(2)$ is guaranteed to find a shortest-path plan to fulfill r_i if one exists. $w(r_i) = 0$ is a special case where r_i can be fulfilled in a single step.

The granularity of an instruction (i.e., rule set $\mathcal{R}$) is then defined as the maximum width across its rules:

$$w(\mathcal{R}) = \max_{r_i \in \mathcal{R}} w(r_i). \quad (1)$$

3.5 Running Example Continued

We continue to demonstrate three instruction sets, each representing a different width level, with two variants visualized in Figure 2. We also append a **counting feature** in Φ to track the number of uncleaned shoes, denoted as N_f .

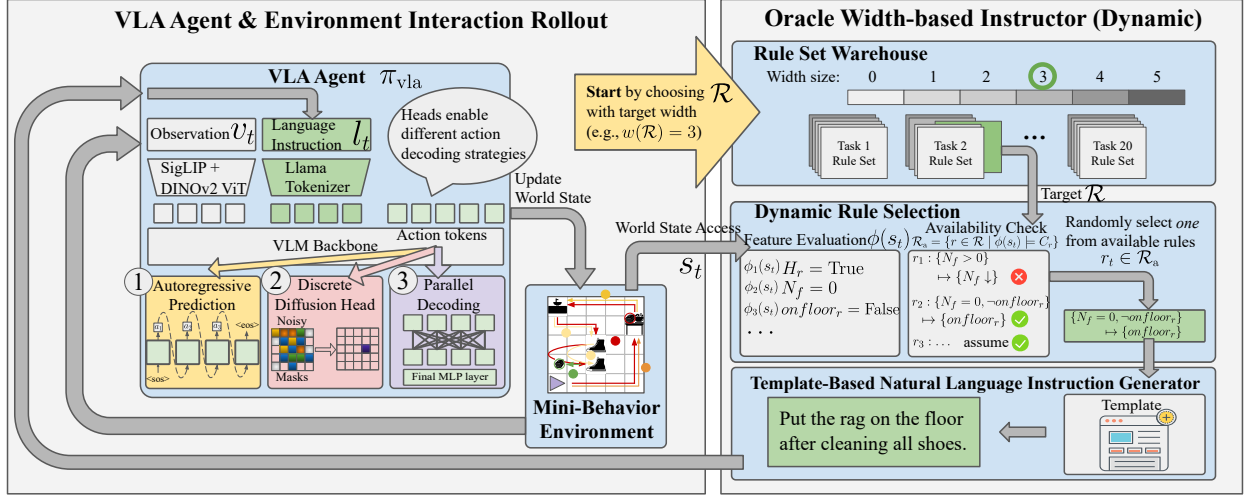


Figure 1: MINI-BEHAVIOR-GRAN Experimental Pipeline. We evaluate the impact of instruction granularity across three VLA action-decoding strategies. During inference time, an Oracle Instructor evaluates the current world state to decide available rule set $\mathcal{R}_a$ and active rule r_t . A rule-based natural language instruction l_t is generated accordingly. The process iterates until completion or max steps.

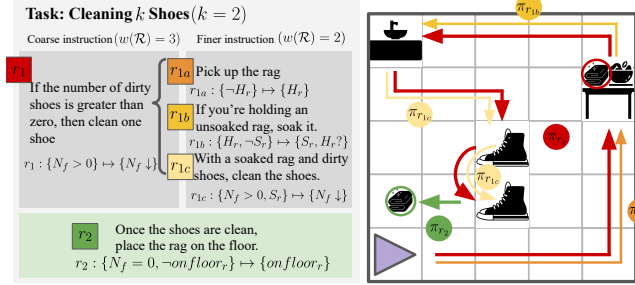


Figure 2: (Left) factoring r_1 into r_{1a} - r_{1c} reduces the planning complexity that the agent must handle internally. More explicit linguistic guidance makes the mapping to action easier to learn. (Right) the long-horizon trajectory for the coarse rule (π_{r_1} , red) is broken into shorter segments by the finer-grained instructions.

Coarse-Grained Instruction ($w(\mathcal{R}) = 3$) This instruction states only the high-level rules and their desired ordering, leaving the internal structure of each sub-task implicit:

- $r_1 : \{N_f > 0\} \mapsto \{N_f \downarrow\}$ (clean a shoe; $w = 3$)
- $r_2 : \{N_f = 0, \neg on_floor_r\} \mapsto \{on_floor_r\}$ (put the rag on the floor after cleaning all shoes; $w = 1$)

Middle-Grained Instruction ($w(\mathcal{R}) = 2$) This variant decomposes r_1 to explicitly state the *soaking* process:

- $r_{1a} : \{\neg H_r\} \mapsto \{H_r\}$ (pick up the rag; $w = 1$)
- $r_{1b} : \{H_r, \neg S_r\} \mapsto \{S_r, H_r\}$ (soak the rag; $w = 2$)
- $r_{1c} : \{N_f > 0, S_r\} \mapsto \{N_f \downarrow\}$ (clean; $w = 2$)
- $r_2 : \{N_f = 0, \neg on_floor_r\} \mapsto \{on_floor_r\}$ (put the rag on the floor; $w = 1$)

This rule set is well-formed: it is goal-terminating via r_2 , and the rules are sequentially reachable, meaning the successor state of any fulfilled rule (that is not the goal) satisfies the condition of at least one rule in $\mathcal{R}$. The set is also robust to

perturbations: if an agent drops the rag after soaking it ($\neg H_r$), the rule set remains applicable by reverting to r_{1a} to reacquire the rag before proceeding to the cleaning task (r_{1c}).

Width Analysis of r_{1b} : From an algorithmic perspective, the width $w(r)$ is the smallest k such that $IW(k)$ solves r . Consider r_{1b} (soaking the rag): if the instruction were expressed as a isolated transition $\{\neg S_r\} \mapsto \{S_r\}$, an agent would need to concurrently coordinate three features: holding the rag (H_r), reaching the sink ($p_s = 0$), and toggling the sink (T_s). However, because r_{1b} 's precondition requires H_r to be established (via r_{1a}), and given that IW search explores states in breadth-first order, any plan that undoes an established feature is necessarily longer than one that preserves it and is subsequently pruned as suboptimal. Hence, the agent can treat H_r as a maintained rather than an actively tracked feature. This reduces the concurrent feature tracking to $\langle T_s, p_s = 0 \rangle$, yielding $w(r_{1b}) = 2$. Although the VLA agent does not perform explicit IW search, this width serves as a rigorous proxy for the inherent planning burden imposed by the instruction.

Fine-grained Instruction ($w = 1$) This variant further decomposes the task into six rules where each transition addresses at most one feature change. The task is fully serialized into three stages: preparation ($r_1 : \{\neg H_r\} \mapsto \{H_r\}$; $r_2 : \{\neg T_s\} \mapsto \{T_s\}$; $r_3 : \{H_r, T_s, \neg S_r\} \mapsto \{S_r\}$); cleaning ($r_4 : \{N_f > 0, \neg H_r, S_r\} \mapsto \{H_r\}$; $r_5 : \{N_f > 0, H_r, S_r\} \mapsto \{N_f \downarrow\}$); and storage ($r_6 : \{N_f = 0, \neg on_floor_r\} \mapsto \{on_floor_r\}$). The higher rule count and denser preconditions is the explicit indicators of increased instruction granularity. This shift displaces the planning complexity to explicit structural textual guidance. The resulting training data exhibits a more direct language-to-action mapping, thereby reducing the grounding gap that VLA agents must learn to bridge.

To communicate these rules to VLA agents, we translate each rule into a natural-language sentence via a template-based generator. These templates verbalize the features within each rule; e.g., $\{H_r, \neg S_r\} \mapsto \{S_r\}$ is realized

as “If you are holding the rag but the rag is not yet soaked, soak the rag.” This construction preserves the granularity differences defined by the rule width. Additional language prompt examples are provided in § E.

4 Mini-BEHAVIOR-Gran Benchmark

To disentangle the influence of instruction granularity in VLA performance, we seek an environment that (1) retains long-horizon and heterogeneous activities so that instruction guidance meaningfully modulates the planning demand, while (2) simplifies perception and low-level control to avoid confounding factors. As such, we use MINI-BEHAVIOR (Jin et al., 2023) as our testbed. Detailed justification for our choice over other environments is provided in § A.

We introduce MINI-BEHAVIOR-GRAN, the first VLA benchmark specifically designed to enable a controlled study and formal quantification of instruction granularity via *rule width*. Specifically, we define a shared, domain-level feature set Φ that captures task-relevant attributes across all 20 complex household tasks on top of the symbolic state abstraction from the original benchmark. For each task, we design rule sets with varying widths (up to six levels). We verify that all rule sets are well-formed and solvable. We utilize an Oracle Instructor to deliver instructions during inference. This ensures that any observed performance drops are strictly attributable to the model’s inability to ground specific granularity levels, rather than stochastic variations in instruction quality or applicability that would confound our cross-granularity analysis (see Figure 1 right panel). At each time t , it evaluates the current feature vector $\phi(s_t)$, identifies the set of available rules $\mathcal{R}_a = \{r \in \mathcal{R} \mid \phi(s_t) \models C_r\}$. We sample an active rule $r_t \in \mathcal{R}_a$ and it is then processed by a template-based translator to generate a natural language instruction l_t , to match the VLA model’s input format. l_t is paired with the current observation v_t and fed into the VLA model to predict action chunk $(a_t, \dots, a_{t+k}) \sim \pi_{\text{vla}}(\cdot \mid v_t, l_t)$.

5 Experiments

5.1 Experimental Setup

Task Suite & Training. We use MINI-BEHAVIOR-GRAN, which contains 20 complex tasks sharing the same feature set Φ . We further classify instruction granularity into three *relative* classes of *Fine* (F), *Medium* (M), and *Coarse* (C) via per-task quantile binning. In most cases, this mapping corresponds to $w = 0$ for Fine, $w = 1$ for Medium, and $w \geq 2$ for Coarse. We evaluate seven training settings: (1) **Pure F/M/C** ($1 : 0 : 0$): trains exclusively on one granularity class; (2) **Centroid** ($1 : 1 : 1$): uniformly sample from all three classes; (3) **Axial F/M/C** ($3 : 1 : 1$): emphasizes one class with some exposure to the rest classes. Our dataset provides 50 training and 10 evaluation instances per task, each paired with reference trajectories for every granularity level generated by BFWS width-based planner from Lipovetzky and Geffner (2017) (see § D for details).

Action-Decoding Archetypes. The primary architectural difference among latest VLA models lies in their action-decoding strategies (Zhong et al., 2025). Hence, we employ

a unified VLM backbone but vary three representative action-decoding strategies (see Figure 1 left panel): (1) **Autoregressive** (AR), exemplified by RT-2 (Zitkovich et al., 2023), OpenVLA (Kim, Pertsch, et al., 2024) and others (J. Huang et al., 2024; O’Neill et al., 2024; Reed et al., 2022) that predict actions sequentially via causal attention; (2) **(Discrete) Diffusion**, which uses diffusion process for action chunk prediction (Bjorck et al., 2025; Black et al., 2025; Shukor et al., 2025). We employ discrete diffusion (Z. Liang et al., 2025) for compatibility with the discrete action space of our testbed; (3) **Parallel Decoding**, utilizing bidirectional attention for single-pass action prediction to improve throughput, as demonstrated by OpenVLA-OFT (Kim, Finn, and P. Liang, 2025), PD-VLA (Song et al., 2025) and other variants (H. Wang et al., 2025; Xiao et al., 2025; Yin et al., 2025). Details on model hyperparameters, hardware and training schedules are in § F.

Research Questions. We address the following questions:

- (Q1) Granularity Trade-offs.** How does instruction granularity impact Success Rate (SR) on within splits?
- (Q2) Generalization.** Can VLA agents trained on certain granularities generalize to other unseen granularities?
- (Q3) Failure Dynamics.** When failing, does the agent get stuck (stagnation) or undo achieved progress (regression)?

5.2 Main Results

We first answer (Q1), short answers as follows:

(A1) VLA performance follows a non-monotonic U-shape, peaking at the Fine and Coarse extremes.

To establish a performance baseline, we first evaluate the models under matched granularity, where training and testing occur on the same instruction class ($F \rightarrow F$, $M \rightarrow M$, $C \rightarrow C$). As shown in Figure 3, all three action-decoding archetypes demonstrate a non-monotonic U-shaped discovery. Note that all instructions are generated via a unified template-based translator that directly verbalizes symbolic rule sets, ensuring uniform vocabulary, syntax, and phrasing across granularity levels and isolating rule width as the causal factor. We interpret this U-shaped curve as follows. When instructions provide a step-by-step mapping to actions (i.e., $w = 0$), the learning burden is minimal, as the agent can directly follow instructions with little reasoning or planning effort. At the coarse end ($w \geq 2$), agents are motivated to develop internal planning skills to ground coarse instructions into actions. Importantly, since each coarse instruction maps to a broader set of visual and action patterns, it helps develop more robust and generalizable language grounding by providing more diverse training data. In contrast, medium instructions ($w = 1$) occupy a sub-optimal grounding regime. They are too sparse to provide a direct language-to-action mapping, yet not coarse enough to foster internal planning skills, thereby resulting in the performance decline.

From a width-based planning perspective, the U-shaped pattern surprises because coarser instructions should monotonically increase planning demands, making learning progressively harder. Yet, a common confounding factor is instruction sequence length (i.e., the number of rule calls required to complete a task), which also impacts SR due to er-

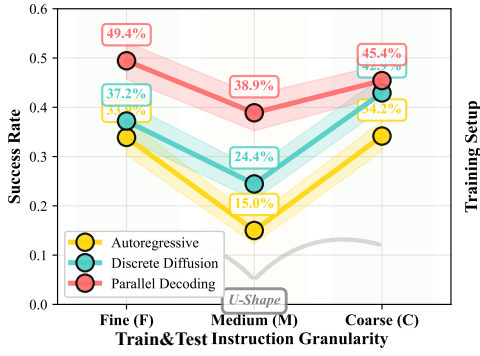


Figure 3: In-Distribution (matched train/test) SR across granularities (i.e., train Fine→test Fine, etc). A U-shaped trend appears.

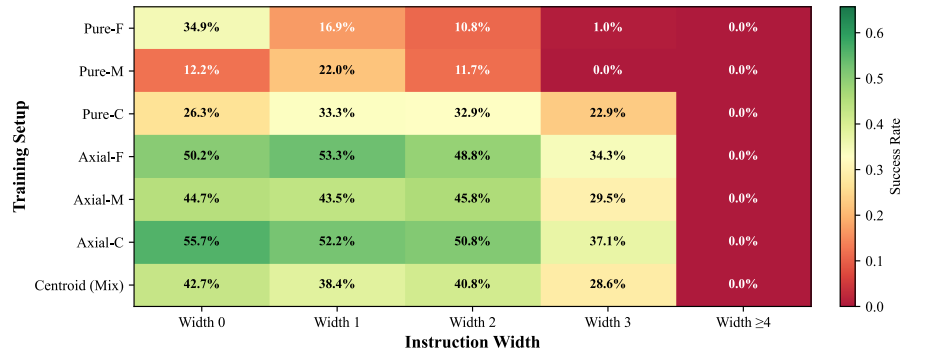


Figure 4: The x-axis are test instruction width, and the y-axis are training setup. Transfer is asymmetric: train coarse → test fine works, but train fine → test coarse does not. Mixed-granularity training improves generalization. *Axial-C* training setting performs best overall.

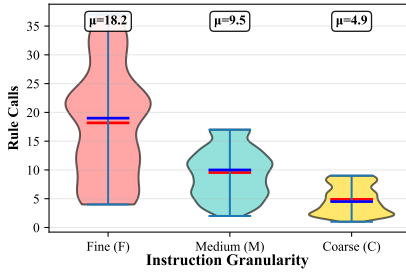


Figure 5: Rule call distribution across instruction granularities. # of rule calls decreases with coarser instructions. It decouples instruction length from width.

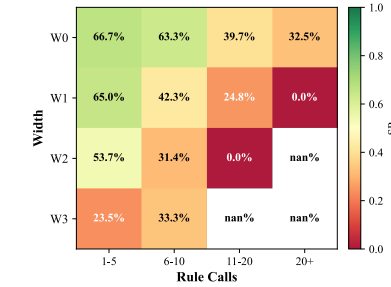


Figure 6: Impact of width and horizon on SR. SR decays with task horizon; larger instruction width imposes an additional burden even when horizon is fixed.

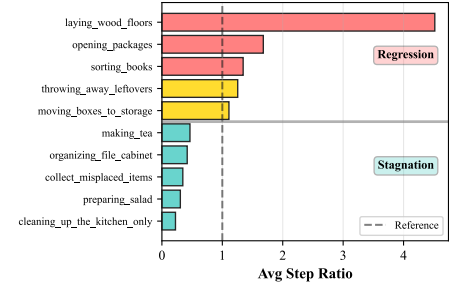


Figure 7: Low-width but long task horizon (e.g., laying woods) tends to cause regression, while high-width tasks tend to stagnate. These contribute to the failure of planning.

ror accumulation. Note that our evaluation uses the same underlying task set across granularities, and thus the reference atomic action sequence length is held constant. The only entangled factor is thus the rule call count. As shown in Figure 5, coarser instructions require fewer rule calls to accomplish the same task, as each instruction maps to a longer action sequence. The medium granularity operates at a moderate number of rule calls. This decouples instruction sequence length from instruction width and confirms the U-shape cannot be attributed to medium-granularity tasks requiring excessively longer instruction sequences or rule calls.

To further decouple the effects of instruction width and sequence length on VLA performance, we analyze Figure 6, obtained by *Centroid* setting as it has balanced exposure to all granularities. The heatmap provides empirical evidence that, for a fixed number of rule calls, SR decreases as instruction width increases. This demonstrates that the width metric, originally developed for symbolic planning, is a meaningful proxy for the difficulty faced by learning-based VLA agents.

Next, we evaluate cross-granularity generalization, where models are trained on one instruction granularity and evaluated on another. Results are summarized in below:

(A2) *Language grounding is asymmetric: models trained on coarse texts generalize to detailed ones, but not vice versa.*

Figure 4 reveals a clear asymmetry: models trained on coarse instructions generalize well to fine ones, demonstrat-

ing robust language grounding that abstracts away from granularity. Conversely, models trained on fine instructions do not generalize to coarse ones, implying a form of grounding that overfits to step-by-step specification. Mixed-granularity training mitigates this issue, with the coarse-weighted *Axial-C* variant yielding the best generalization overall.

For (Q3), the failure mode is dependent on task structure: (A3) *Low-width but long-horizon tasks tend to cause regression, while high-width tasks tend to stagnate.*

We interpret these failure modes through the lens of width-based planning (see Figure 7). In high-regression tasks, which often require manipulating multiple objects toward a collective goal, agents frequently exhibit subgoal serialization failures. In these settings, the model fails to maintain satisfied fluents over time, cycling back to earlier states. Conversely, stagnation in high-width tasks (e.g., making tea) reflects a width-planning bottleneck: as simultaneous feature dependencies increase ($w \geq 4$), agents struggle to track multiple simultaneous feature dependencies to enable a valid state progression. More analysis is in § G.

6 Dilemma of Coarse Instruction Training

Our analyses in (Q1) and (Q3) suggest a practical advantage for training with coarse instructions: models maintain strong in-distribution performance while generalizing effectively to unseen, finer-grained instructions. To further evaluate this

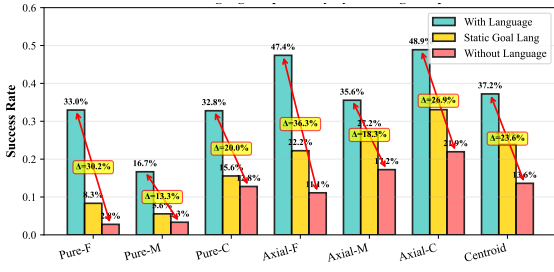


Figure 8: Both mixed and coarse-only training “generalize” better in novel instruction settings (smaller SR gap).

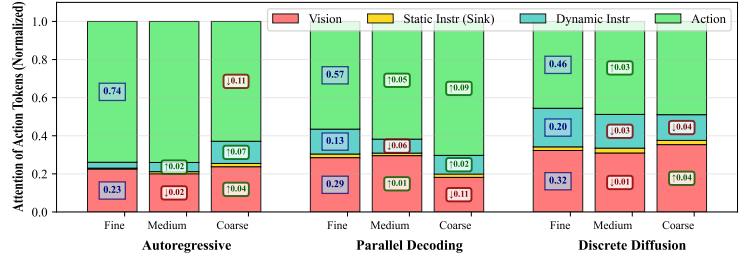


Figure 9: Attention from action tokens. Self-attention dominates, followed by vision; No clear trend across granularity levels.

generalization capability, we test robustness to more extreme instruction variations: providing only a static goal description (fixed for the entire episode, akin to the LIBERO testbed (Liu et al., 2023)) and providing no instruction at all.

Results are shown in Figure 8. Mixed-granularity training, particularly the *Axial-C* variant, achieves the highest SR when presented with the unseen static-goal descriptions. More strikingly, when the language input is entirely absent, *Axial-C* maintains a non-trivial SR of 21%. This raises a fundamental question: does the success of coarse instruction training stem from improved generalization via robust language grounding, or from “language ignoring”, where the policy becomes invariant to language inputs?

Quantifying Language Ignoring. “Language ignoring” is a phenomenon recently identified in VLA training, where agents appear to over-rely on visual patterns, making their policies invariant to language input (Fei et al., 2025; Xing et al., 2025). A standard metric for this is $\Delta\text{SR} = \text{SR}_{\text{w/ lang}} - \text{SR}_{\text{w/o lang}}$, where a smaller gap indicates stronger ignoring. Our data reveals a trend: models trained purely on coarse instructions (*Pure-C*) exhibit a smaller ΔSR (20.0%) than those trained on fine instructions (*Pure-F*), suggesting coarser training may induce greater language ignoring.

Attention Analysis. To probe the model’s internal reliance on language more directly, we analyze the attention distribution from action tokens to the vision, language, and self-modalities (see Figure 9). Contrary to a simple visual-shortcut hypothesis, we find no clear trend of decreasing attention to the language modality as instruction granularity coarsens. Across all training regimes, self-attention receives the largest share, followed by vision, with language consistently receiving the smallest portion. Notably, autoregressive models attend least to language, which could explain their lower overall SR in Figure 3.

Why Coarse Instructions Encourage Language Ignoring? Xu et al. (2025) posit a causal mechanism: if the language instruction l is highly predictable from visual context v (i.e., large $P(l|v)$), a multimodal model may learn to bypass language processing to save computational energy. This aligns with our benchmark, where expert trajectories are fixed but described at varying granularities: consider a surjective mapping f from a fine-grained label space $L_{\text{fine}} = l_1, \dots, l_n$ to a coarser one $L_{\text{coarse}} = c_1, \dots, c_m$ with $m < n$. The conditional probability for a coarse label aggregates over its constituent fine labels: $P(c_j|v) = \sum_{l_i \in f^{-1}(c_j)} P(l_i|v)$.

Since the same total probability mass is distributed over fewer coarse categories, the average $P(c_j|v)$ is higher. According to the theory, this increased predictability could encourage language ignoring for coarsely-trained models.

Mixed-granularity training mitigates this effect by increasing the diversity of the language space L seen during training, which lowers the average $P(l|v)$. We thus see all mixed-granularity models exhibit larger ΔSR gaps than their pure counterparts, indicating reduced language ignoring. Among these, the fine-weighted mixture *Axial-F* emerges as a particularly balanced choice, achieving a success rate comparable to *Axial-C* while maintaining a substantially larger ΔSR .

Our analysis provides a complementary perspective on the language ignoring phenomenon, demonstrating its variation across instruction granularities, a factor not explored in prior work. The trade-off between final performance and language reliance highlights an open challenge in VLA design: how to structure training for both robust generalization and faithful language grounding.

7 Conclusion and Future Work

This work formalizes and systematically studies how instruction granularity impacts the performance and generalization of VLA agents. Our controlled analysis shows that VLA performance is highly sensitive to granularity, exhibiting a non-monotonic U-shaped relationship that suggests real-world systems may benefit from deliberate instruction granularity design. We provide a granularity-centric diagnosis of the “language ignoring” phenomenon, linking it to instruction predictability and identifying mixed-granularity training as an effective mitigation strategy.

Future Work. Several promising directions follow. First, moving beyond symbolic, grid-based domains to estimate *planning width* in complex, continuous environments would greatly enhance generalizability. This requires developing methods to approximate *width* from ambiguous observations and diverse instruction formats. Second, to enable this, a key challenge is the automatic derivation of relevant feature sets. Future work could develop learnable feature modules that dynamically extract or construct planning-relevant state abstractions directly from multimodal inputs (pixels, knowledge bases, or language descriptions), reducing reliance on hand-designed symbolic predicates and bridging the gap between vectorized data and structured planning models.

Use of LLM Statement An LLM was used only for polish-
ing language; the scientific content is entirely original.

References

- Akula, Arjun R. et al. (2022). “ALFRED-L: Investigating the Role of Language for Action Learning in Interactive Visual Environments”. In: *EMNLP*. Association for Computational Linguistics, pp. 9369–9378.
- Bjorck, Johan et al. (2025). “Gr00t n1: An open foundation model for generalist humanoid robots”. In: *arXiv preprint arXiv:2503.14734*.
- Black, Kevin et al. (27–30 Sep 2025). “ $\pi_{0.5}$: a Vision-Language-Action Model with Open-World Generalization”. In: *Proceedings of The 9th Conference on Robot Learning*. Ed. by Joseph Lim, Shuran Song, and Hae-Won Park. Vol. 305. Proceedings of Machine Learning Research. PMLR, pp. 17–40. URL: <https://proceedings.mlr.press/v305/black25a.html>.
- Bonet, Blai, Guillem Francès, and Hector Geffner (2019). “Learning Features and Abstract Actions for Computing Generalized Plans”. In: *AAAI*. AAAI Press, pp. 2703–2710.
- Bonet, Blai and Hector Geffner (2021). “General Policies, Representations, and Planning Width”. In: *AAAI*. AAAI Press, pp. 11764–11773.
- Clark, Herbert H. (1996). *Using Language*. “Using” Linguistic Books. Cambridge University Press.
- Dong, Li and Mirella Lapata (July 2018). “Coarse-to-Fine Decoding for Neural Semantic Parsing”. In: *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Ed. by Iryna Gurevych and Yusuke Miyao. Melbourne, Australia: Association for Computational Linguistics, pp. 731–742. DOI: 10.18653/v1/P18-1068. URL: <https://aclanthology.org/P18-1068/>.
- Drexler, Dominik, Jendrik Seipp, and Hector Geffner (2022). “Learning Sketches for Decomposing Planning Problems into Subproblems of Bounded Width”. In: *ICAPS*. AAAI Press, pp. 62–70.
- (2024). “Expressing and Exploiting Subgoal Structure in Classical Planning Using Sketches”. In: *J. Artif. Intell. Res.* 80.
- Fei, Senyu et al. (2025). *LIBERO-Plus: In-depth Robustness Analysis of Vision-Language-Action Models*. arXiv: 2510.13626 [cs.RO]. URL: <https://arxiv.org/abs/2510.13626>.
- Glossop, Catherine et al. (2025). *CAST: Counterfactual Labels Improve Instruction Following in Vision-Language-Action Models*. arXiv: 2508.13446 [cs.RO]. URL: <https://arxiv.org/abs/2508.13446>.
- Huang, Chi-Pin et al. (2025). *ThinkAct: Vision-Language-Action Reasoning via Reinforced Visual Latent Planning*. arXiv: 2507.16815 [cs.CV]. URL: <https://arxiv.org/abs/2507.16815>.
- Huang, Hui et al. (2025). “MuSC: Improving Complex Instruction Following with Multi-granularity Self-Contrastive Training”. In: *ACL (1)*. Association for Computational Linguistics, pp. 10667–10686.
- Huang, Jiangyong et al. (2024). “An Embodied Generalist Agent in 3D World”. In: *ICML*.
- Jin, Emily et al. (2023). “Mini-BEHAVIOR: A Procedurally Generated Benchmark for Long-horizon Decision-Making in Embodied AI”. In: *NeurIPS 2023 Workshop on Generalization in Planning*. URL: <https://openreview.net/forum?id=Ghl9pYaVh5>.
- Karnik, Sathwik et al. (2025). *Embodied Red Teaming for Auditing Robotic Foundation Models*. arXiv: 2411.18676 [cs.RO]. URL: <https://arxiv.org/abs/2411.18676>.
- Kim, Moo Jin, Chelsea Finn, and Percy Liang (2025). “Fine-Tuning Vision-Language-Action Models: Optimizing Speed and Success”. In: *arXiv preprint arXiv:2502.19645*.
- Kim, Moo Jin, Karl Pertsch, et al. (2024). “OpenVLA: An Open-Source Vision-Language-Action Model”. In: *CoRL*. Vol. 270. Proceedings of Machine Learning Research. PMLR, pp. 2679–2713.
- Lachmy, Royi et al. (2022). “Draw Me a Flower: Processing and Grounding Abstraction in Natural Language”. In: *Trans. Assoc. Comput. Linguistics* 10, pp. 1341–1356.
- Li, Chengshu, Ruohan Zhang, et al. (2022). “BEHAVIOR-1K: A Benchmark for Embodied AI with 1, 000 Everyday Activities and Realistic Simulation”. In: *CoRL*. Vol. 205. Proceedings of Machine Learning Research. PMLR, pp. 80–93.
- Li, Wei et al. (2025). “CogVLA: Cognition-Aligned Vision-Language-Action Model via Instruction-Driven Routing & Sparsification”. In: *Advances in neural information processing systems*.
- Li, Xuanlin et al. (2024). “Evaluating Real-World Robot Manipulation Policies in Simulation”. In: *CoRL*. Vol. 270. Proceedings of Machine Learning Research. PMLR, pp. 3705–3728.
- Li, Zechang et al. (2020). “Domain Adaptation for Semantic Parsing”. In: *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence, IJCAI 2020*. Ed. by Christian Bessiere. ijcai.org, pp. 3723–3729. DOI: 10.24963/IJCAI.2020/515. URL: <https://doi.org/10.24963/ijcai.2020/515>.
- Liang, Zhixuan et al. (2025). “Discrete diffusion vla: Bringing discrete diffusion to action decoding in vision-language-action policies”. In: *arXiv preprint arXiv:2508.20072*.
- Lipovetzky, Nir and Hector Geffner (2012). “Width and Serialization of Classical Planning Problems”. In: *ECAI*. Vol. 242. Frontiers in Artificial Intelligence and Applications. IOS Press, pp. 540–545.
- (2017). “Best-First Width Search: Exploration and Exploitation in Classical Planning”. In: *AAAI*. AAAI Press, pp. 3590–3596.
- Liu, Bo et al. (2023). “LIBERO: Benchmarking Knowledge Transfer for Lifelong Robot Learning”. In: *NeurIPS*.
- Ma, Yuen et al. (2025). *A Survey on Vision-Language-Action Models for Embodied AI*. arXiv: 2405.14093 [cs.RO]. URL: <https://arxiv.org/abs/2405.14093>.
- Mu, Yao et al. (2025). “RoboTwin: Dual-Arm Robot Benchmark with Generative Digital Twins”. In: *CVPR*. Computer Vision Foundation / IEEE, pp. 27649–27660.

- Myers, Vivek et al. (2024). “Policy Adaptation via Language Optimization: Decomposing Tasks for Few-Shot Imitation”. In: *CoRL*. Vol. 270. Proceedings of Machine Learning Research. PMLR, pp. 1402–1426.
- Norman, Donald A. (2002). *The Design of Everyday Things*. USA: Basic Books, Inc. ISBN: 9780465067107.
- O’Neill, Abby et al. (2024). “Open X-Embodiment: Robotic Learning Datasets and RT-X Models : Open X-Embodiment Collaboration”. In: *ICRA*. IEEE, pp. 6892–6903.
- Reed, Scott E. et al. (2022). “A Generalist Agent”. In: *Trans. Mach. Learn. Res.* 2022.
- Shi, Lucy Xiaoyang et al. (2025). “Hi Robot: Open-Ended Instruction Following with Hierarchical Vision-Language-Action Models”. In: *ICML*.
- Shridhar, Mohit et al. (2020a). “ALFRED: A Benchmark for Interpreting Grounded Instructions for Everyday Tasks”. In: *CVPR*. Computer Vision Foundation / IEEE, pp. 10737–10746.
- (2020b). “Alfred: A benchmark for interpreting grounded instructions for everyday tasks”. In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 10740–10749.
- Shukor, Mustafa et al. (2025). “Smolvla: A vision-language-action model for affordable and efficient robotics”. In: *arXiv preprint arXiv:2506.01844*.
- Song, Wenxuan et al. (2025). “PD-VLA: Accelerating Vision-Language-Action Model Integrated with Action Chunking via Parallel Decoding”. In: *IROS*. IEEE, pp. 13162–13169.
- Sussman, Gerald Jay (1975). *A computer model of skill acquisition*. Elsevier Science Inc.
- Wang, Hongyu et al. (2025). “BitVLA: 1-bit Vision-Language-Action Models for Robotics Manipulation”. In: *arXiv preprint arXiv:2506.07530*.
- Wing, Jeannette M. (2010). “Computational Thinking: What and Why?” In: *The Link*. URL: www.cs.cmu.edu.
- Xiao, Lei et al. (2025). “AVA-VLA: Improving Vision-Language-Action models with Active Visual Attention”. In: *arXiv preprint arXiv:2511.18960*.
- Xing, Youguang et al. (2025). “Shortcut Learning in Generalist Robot Policies: The Role of Dataset Diversity and Fragmentation”. In: *9th Annual Conference on Robot Learning*. URL: <https://openreview.net/forum?id=PTJopl2uaU>.
- Xu, Kechun et al. (2025). *Seeing to Act, Prompting to Specify: A Bayesian Factorization of Vision Language Action Policy*. arXiv: 2512.11218 [cs.LG]. URL: <https://arxiv.org/abs/2512.11218>.
- Yang, Ruochu, Fumin Zhang, and Mengxue Hou (2024). “OceanPlan: Hierarchical Planning and Replanning for Natural Language AUV Piloting in Large-scale Unexplored Ocean Environments”. In: *WUWNet*. ACM, 11:1–11:5.
- Yang, Shuai et al. (2025). *InstructVLA: Vision-Language-Action Instruction Tuning from Understanding to Manipulation*. arXiv: 2507.17520 [cs.LG]. URL: <https://arxiv.org/abs/2507.17520>.
- Yin, Cheng et al. (2025). “DeepThinkVLA: Enhancing Reasoning Capability of Vision-Language-Action Models”. In: *arXiv preprint arXiv:2511.15669*.
- Zheng, Huaixiu Steven et al. (2024). “Take a Step Back: Evoking Reasoning via Abstraction in Large Language Models”. In: *ICLR*.
- Zhong, Yifan et al. (2025). “A Survey on Vision-Language-Action Models: An Action Tokenization Perspective”. In: *arXiv preprint arXiv:2507.01925*.
- Zitkovich, Brianna et al. (2023). “RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control”. In: *CoRL*. Vol. 229. Proceedings of Machine Learning Research. PMLR, pp. 2165–2183.

Technical Appendix

Appendix Table of Contents

1. **Anticipatory defense on the choice of test environments and metrics (§ A)**
Justification for using MINI-BEHAVIOR over alternatives (e.g., iGibson, CALVIN) based on controllability, symbolic abstraction, and isolation of language-grounding challenges.
2. **Symbolic Representation of the Embodied Task (§ B)** Formal definition of states, actions, and planning problems using classical STRIPS/PDDL semantics.
3. **Details of the MINI-BEHAVIOR Environment (§ C)** Overview of scene layout, object types, action space, and perception model.
4. **Details of the MINI-BEHAVIOR-GRAN Extension (§ D)** Procedure for generating multi-granularity instructions, details on the feature set, oracle instructor, and dataset construction.
5. **Rule Set and Natural Language Prompt Examples (§ E)** Details of the rule set bank of 20 tasks and sample generated instructions at different granularities.
6. **Implementation Details (§ F)** Model architecture (Prismatic-7B backbone, SigLIP+DINOv2 visual encoders), training hyperparameters, optimizer settings.
7. **Additional Experimental Results (§ G)** Extended analysis and ablations.

A Anticipatory defense on the choice of test environments and metrics

In this work, we selected Mini-BEHAVIOR as our testbed. This choice is motivated by several key considerations aligned with our research objectives. Our research objective is not to solve the full stack of robotic embodiment (which includes perception, continuous control, and physics), but specifically to isolate and analyze the impact of instruction granularity on language grounding and long-horizon planning.

Therefore, high-fidelity simulators (e.g., iGibson (C. Li, Xia, et al., 2022), BEHAVIOR-1K (C. Li, R. Zhang, et al., 2022)) introduce significant noise through complex texture rendering and continuous motor control. While realistic, these factors act as confounders. When an agent fails in such environments, it is often difficult to attribute the failure to a lack of linguistic understanding (granularity issues) or a failure in low-level execution (e.g., grasping physics or occlusion). Mini-BEHAVIOR retains the high-level decision complexity of household tasks—long horizons, multi-object interactions, and state dependencies—while abstracting away low-level sensorimotor noise.

In the perspective of fast prototyping and iterative experimentation, we have to also consider the rendering and physics simulation costs. Mini-BEHAVIOR’s grid-based discrete control and simplified visual representation allow for rapid data generation and model training.

Another consideration is the diversity of the task structures so that we can systematically vary instruction granularity across a wide range of scenarios. As such, Crafter (Hafner, 2021), which focuses on survival in a single domain, lacks the diverse task structures needed to obtain robust insights about instruction granularity.

Table 2 provides a detailed comparison of Mini-BEHAVIOR against other prominent benchmarks, highlighting why they were less suitable for this specific investigation.

Table 2: Comparison of Embodied AI Environments regarding their suitability for studying Instruction Granularity.

Environment	Control	Horizon	Visuals	Symbolic Abstraction	Limitation
Mini-BEHAVIOR ^(*)	Discrete	Long	Grid	Support	Ideal balance: Supports symbolic state abstraction; diverse task suite; removes control noise while retaining logic complexity.
Crafter	Discrete	Long	Cartoon	Partial	Single task suite (survival); partially observable (memory heavy); few object types.
Meta-World	Continuous	Short	3D	No	Continuous control; no symbolic abstraction; focus on multi-task RL/manipulation, not long-horizon planning.
LIBERO	Continuous	Short	3D	No	Robotic arm focus; continuous control; no symbolic abstraction; VLA success rate already saturated (tasks relatively easy).
RoboTwin 2.0	Continuous	Short	3D	No	Continuous control; no symbolic abstraction; dual-arm manipulation focus; hard to segment expert trajectories and construct granular instructions.
CALVIN	Continuous	Medium	3D	No	Hard to design instruction granularity due to continuous action space (though provides diverse language); confounds planning with control difficulties.
ALFRED	Discrete	Long	3D Photo	Support	3D partially observable; poor infrastructure support for VLA training; high rendering cost.
iGibson / BEHAVIOR-1K	Continuous	Long	3D Photo	Partial	Good for long-horizon tasks, but 3D realistic vision context becomes a confounder

B Symbolic Representation of the Embodied Task

We formalize the Mini-BEHAVIOR environment as a deterministic grid-based symbolic world. This allows us to represent the challenge as a classical planning problem $\mathcal{P} = \langle \mathcal{F}, \mathcal{A}, s_0, G \rangle$, where:

- $\mathcal{F}$ is a finite set of propositional fluents representing the state variables of the environment (e.g., object locations, states like *cooked* or *open*).
- $\mathcal{A}$ is a finite set of actions available to the agent.
- $s_0 \subseteq \mathcal{F}$ is the initial state configuration.
- $G \subseteq \mathcal{F}$ is the goal condition that must be satisfied.

A state $s \in \mathcal{S}$ is defined as a truth valuation over $\mathcal{F}$, with the state space $\mathcal{S} \subseteq 2^{\mathcal{F}}$ encompassing all valid combinations of fluent truth values.

C Details of Mini-BEHAVIOR Environment

To operationalize our formalism, we developed a **Universal PDDL Domain Model** (available at `minibehavior_gran_domain_model.pddl` in the codebase) capable of supporting all 20 household tasks without modification. This unified model comprises over 1,000 lines of PDDL 2.1 definitions and is fully compatible with the LAMA planner. We highly recommend checking the codebase for the complete domain model.

Table 3: Planning complexity in typical tasks.

Metric	Typical Range
Objects	15–40
Grid Locations	20–60
Ground Actions	10^3 – 10^5
Plan Length	50–200 primitive actions
Planning Time (LAMA)	0.5–300+ seconds

Search Space Characteristics

D Details of MINI-BEHAVIOR-GRAN Extension

MINI-BEHAVIOR provides the foundational task skeletons, which we extend in MINI-BEHAVIOR-GRAN to support multi-granularity instruction generation. A core component of this extension is the definition of a symbolic feature set used to generate rule-based policy sketches.

D.1 Feature Categories

The environment defines two primary categories of features: Boolean State Features and Quantitative Features.

Boolean State Features

These features evaluate whether a specific object instance satisfies a predicate. They require grounding (binding to specific object instances, e.g., `rag-01`).

Table 4: Boolean State Features

Feature	Applicable Objects	Semantics
Holding	All Items	Is the agent holding the specified object?
isOpen	Cabinets, Drawers, Doors	Is the container/door open?
isToggled	Sinks, Stoves, Switches	Is the appliance turned on?
isSoaked	Rags, Sponges	Is the cleaning tool wet?
isCleaned	Shoes, Dishes, Floor	Is the object free of stains?
isDustFree	Furniture, Surfaces	Is the object free of dust?

Quantitative and Spatial Features

These features capture distances, counts, and spatial relationships. Crucially, they support temporal comparison allowing conditions such as “distance decreased” or “count dropped to zero.”

- **DistanceToNearest(*type*, *condition*)**: Measures the Manhattan distance from the agent to the nearest object of a specific type.

– *Conditions*: =0 (adjacent), >0, increase, decrease, change.

- **Count(*type*, *condition*, *function*)**: Counts the number of objects of a specific type that satisfy a filter function.

– *Conditions*: =0, >0, increase, decrease.

D.2 Count Functions

A diverse set of counting functions is implemented to support complex logical conditions (e.g., “all shoes are clean” is equivalent to `count_not_cleaned == 0`).

Table 5: Available Count Functions for Rule Definitions

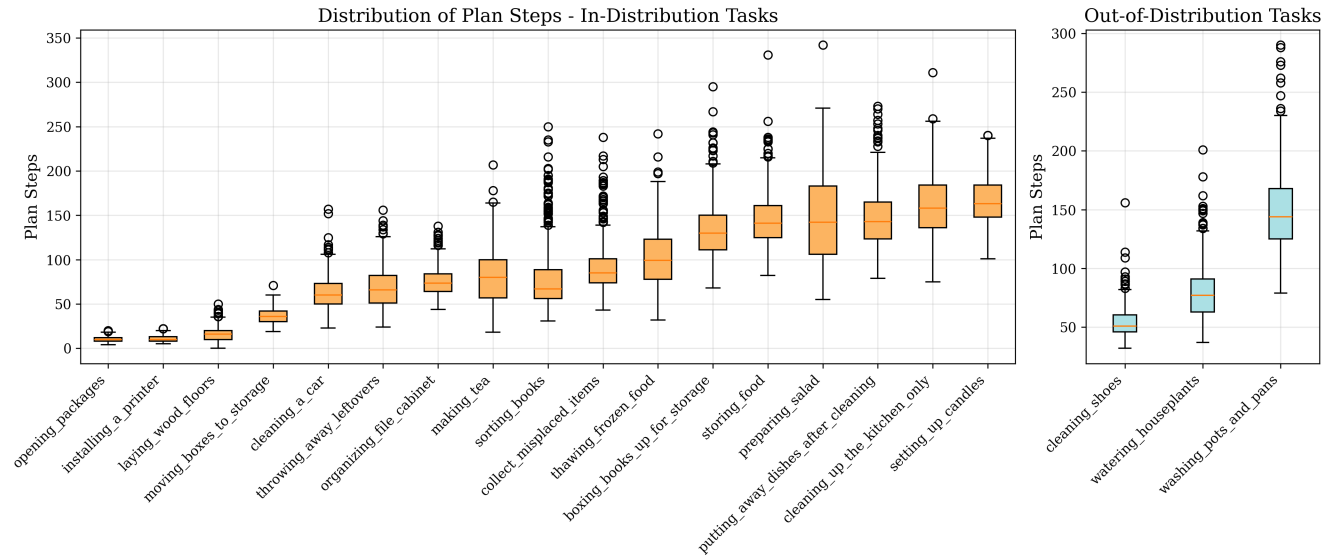
Category	Function Description
State-Based	count_not_cleaned: Objects that are currently stained. count_not_soaked: Cleaning tools that are dry. count_not_toggled: Appliances that are turned off. count_closed: Containers/doors that are closed. count_not_sliced: Food items that require slicing.
Spatial	count_not_onfloor: Objects not placed on the floor layer. count_isolated: Objects with no neighbors of the same type. count_not_inside_target: Objects not contained within a specific furniture. count_not_ontop: Objects not placed on top of a specific surface. count_not_near_target: Objects not within 1 step of a target type.
Task-Specific	count_incomplete_salad: Plates lacking necessary ingredients. count_not_complete_tables: Tables lacking specific setups (e.g., candles).

D.3 Reference Plan Generation and Task Complexity Analysis

858

While the original MINI-BEHAVIOR framework provides the high-level skeletons for household activities, it does not provide reference plans by default. Thus, we utilize the BWFS (Best-First Width Search) classical planner to generate reference plans for all 20 tasks in MINI-BEHAVIOR-GRAN. These reference plans serve as the ground truth for our agents and allow us to quantitatively categorize the complexity of each task.

859
860
861
862



Task	Split	Class	Goal Description	Mean Steps
opening_packages	ID	short	Open every package.	10.2
installing_a_printer	ID	short	Place a printer on a table and turn it on.	10.6
laying_wood_floors	ID	short	Lay every plywood sheet on the floor, each touching at least one other sheet.	15.9
moving_boxes_to_storage	ID	short	Place every carton inside a shelf.	36.3
cleaning_a_car	ID	short	Make at least one car dust-free. Place a rag and soap together inside a bucket.	62.6
throwing_away_leftovers	ID	middle	Throw away every hamburger by placing it in a trash can.	68.3
organizing_file_cabinet	ID	middle	Put a marker on a table and file every folder and document in a cabinet.	75.5
making_tea	ID	middle	Slice a lemon. Put a teapot on an activated stove. Keep a soaked teabag with the teapot.	78.7
sorting_books	ID	middle	Place every book and every hardback on a shelf.	78.8
collect_misplaced_items	ID	middle	Place all gym shoes, necklaces, notebooks, and socks on a table.	90.7
thawing_frozen_food	ID	middle	Place a date label next to a fish. Put every fish next to a sink. Set an olive next to a sink.	101.7
boxing_books_up_for_storage	ID	long	Place every book inside a box.	134.4
storing_food	ID	long	Store every cookable item inside a cabinet.	144.6
preparing_salad	ID	long	Slice all apples and tomatoes; place them on plates. Put every radish and lettuce on plates. Ensure each plate holds at least one cookable item.	146.7
putting_away_dishes_after_cleaning	ID	long	Put every plate inside a cabinet.	147.7
cleaning_up_the_kitchen_only	ID	long	Put every blender on a countertop. Refrigerate all apples and casseroles. Keep soap next to a sink. Make plates unstained and cabinets dust-free. Place each rag either beside or inside the sink. Store plates and vegetable oil in different cabinets.	161.0
setting_up_candles	ID	long	On every table, place three distinct candles on top.	165.2
cleaning_shoes	OOD	short	Leave a towel on the floor and make sure every shoe is unstained and dust-free.	54.2
watering_houseplants	OOD	middle	Water every potted plant until it is soaked.	79.4
washing_pots_and_pans	OOD	long	Clean every teapot, kettle, and pan so they're unstained, then store each inside a cabinet.	149.5

Table 6: Classification of tasks in MINI-BEHAVIOR-GRAN based on the median number of plan steps, as determined by the BWFS planner. Tasks are categorized into short, middle, and long classes to reflect varying planning complexity based on plan length.

Detailed statistics regarding the specific rule sets used for instruction generation and the resulting natural language statistics are discussed in the next section.

E Rule Set and Natural Language Prompt Examples For Different Tasks

We present the width-0 rule set for all 20 household tasks in the Mini-BEHAVIOR-Gran environment. For $w > 0$, please refer to the codebase at `mini-behavior-gran/mini_behavior/utis/policy_sketch/`.

E.1 Task 1: Laying Wood Floors

Features:

- H : holding an isolated wood (plywood)
- p : distance to the nearest isolated wood
- t : distance to an arbitrary (non-held) wood we'll place next to
- n : number of isolated woods

Width-0 Sketch:

$\{\neg H, p > 0, n > 0\} \mapsto \{p \downarrow, t?\}$	; move close to isolated wood
$\{\neg H, p = 0, n > 0\} \mapsto \{H\}$	; pick up isolated wood when reachable
$\{H, t > 0, n > 0\} \mapsto \{t \downarrow\}$	; move to some wood location
$\{H, n > 0, t = 0\} \mapsto \{H?, n \downarrow, p?\}$	; drop the isolated wood next to other wood

E.2 Task 2: Preparing Salad

Features:

- H_n : holding a not-sliceable food
- H_s : holding a sliceable food

- H_k : holding a slicer
- U : number of sliceable food that is not sliced yet
- d_p : distance to the selected incomplete plate p
- d_n, d_s : distances to nearest not-sliceable / sliceable
- k : distance to a slicer (knife)
- B : selected plate p complete bottom salad
- n : number of incomplete plates
- nn : number of plates that do not have bottom salad yet

Width-0 Sketch:

Acquire slicer:

$\{U > 0, \neg H_k, k > 0\} \mapsto \{k \downarrow\}$	; move to slicer
$\{U > 0, \neg H_k, k = 0\} \mapsto \{H_k\}$	; hold a slicer
$\{U > 0, H_k, d_s > 0\} \mapsto \{d_s \downarrow\}$	; move to sliceable food
$\{U > 0, H_k, d_s = 0\} \mapsto \{U \downarrow, \neg H_k\}$	; cut sliceable food
$\{U = 0, H_k\} \mapsto \{\neg H_k\}$	; put down slicer

Place bottom salad:

$\{n > 0, U = 0, nn > 0, \neg B, \neg H_n, d_n > 0\} \mapsto \{d_n \downarrow\}$
$\{n > 0, U = 0, nn > 0, \neg B, \neg H_n, d_n = 0\} \mapsto \{H_n\}$
$\{n > 0, U = 0, nn > 0, \neg B, H_n, d_p > 0\} \mapsto \{d_p \downarrow\}$
$\{n > 0, U = 0, nn > 0, \neg B, H_n, d_p = 0\} \mapsto \{B, nn \downarrow, \neg H_n\}$

Handle sliceable salad:

$\{nn = 0, n > 0, U = 0, \neg H_s, d_s > 0\} \mapsto \{d_s \downarrow\}$	; move to sliceable food
$\{nn = 0, n > 0, U = 0, \neg H_s, d_s = 0\} \mapsto \{H_s\}$	; pick up sliceable food
$\{nn = 0, n > 0, U = 0, H_s, d_p > 0\} \mapsto \{d_p \downarrow\}$	; move to the plate
$\{nn = 0, n > 0, U = 0, H_s, d_p = 0\} \mapsto \{n \downarrow, \neg H_s\}$	; put down sliced food

E.3 Task 3: Cleaning Up the Kitchen Only

Features:

- *Holding flags*: H_b (blender), H_f (food), H_s (soap), H_r (rag), H_p (plate), H_o (vegetable oil)
- *Distances*: d_{ct} (countertop), d_{fr} (fridge), d_{si} (sink), d_{c1} (cabinet for plate), d_{c2} (cabinet for oil), d_x (to object x)
- *State booleans*: $\text{Open}(c)$, $\text{Tog}(s)$, Soaked , $\text{DustFree}(c)$, $\text{Clean}(p)$, $\text{OilIn}(c_2)$, Diff
- *Counters*: N_b (blenders not on countertop), N_a (apples not in fridge), N_{cas} (casseroles not in fridge), N_s (soaps not next to sink), N_{cab} (cabinets not dusted), N_r (rags not next to sink), N_{p_clean} (plates not clean), N_{oil} (oil not in cabinet), N_{p_store} (plates not stored), N_{close} (closed furniture)

Width-0 Sketch:

Open Cabinet and Fridge:

$\{N_{close} > 0, d_c > 0\} \mapsto \{d_c \downarrow\}$	; move toward closed furniture
$\{N_{close} > 0, d_c = 0\} \mapsto \{N_{close} \downarrow\}$	

Oil placement:

$\{N_{close} = 0, \neg H_o, d_{oil} > 0\} \mapsto \{d_{oil} \downarrow\}$	; approach oil
$\{N_{close} = 0, \neg H_o, d_{oil} = 0\} \mapsto \{H_o\}$	; pick up oil
$\{N_{close} = 0, H_o, N_{oil} > 0, d_{c2} > 0\} \mapsto \{d_{c2} \downarrow\}$	; go to cabinet c_2
$\{N_{close} = 0, H_o, N_{oil} > 0, d_{c2} = 0\} \mapsto \{N_{oil} \downarrow, \neg H_o\}$	; place oil in c_2

902 *Plate cleaning:*

$$\begin{aligned}
& \{\neg \text{Tog}(s), d_{si} > 0\} \mapsto \{d_{si} \downarrow\} && ; \text{toggle sink on} \\
& \{\neg \text{Tog}(s), d_{si} = 0\} \mapsto \{\text{Tog}(s)\} \\
& \{\neg H_r, d_{rag} > 0\} \mapsto \{d_{rag} \downarrow\} && ; \text{get rag} \\
& \{\neg H_r, d_{rag} = 0\} \mapsto \{H_r\} \\
& \{H_r, \neg \text{Soaked}, d_{si} > 0\} \mapsto \{d_{si} \downarrow\} && ; \text{soak rag} \\
& \{H_r, \neg \text{Soaked}, d_{si} = 0\} \mapsto \{\text{Soaked}\} \\
& \{H_r, \text{Soaked}, N_{p_clean} > 0, d_{plate} > 0\} \mapsto \{d_{plate} \downarrow\} \\
& \{H_r, \text{Soaked}, N_{p_clean} > 0, d_{plate} = 0\} \mapsto \{N_{p_clean} \downarrow, d_{plate}?\}
\end{aligned}$$

903 *Plate placement distinct from oil:*

$$\begin{aligned}
& \{N_{close} = 0, N_{oil} = 0, N_{p_clean} = 0, \neg H_p, N_{p_store} > 0, d_{plate} > 0\} \mapsto \{d_{plate} \downarrow\} \\
& \{N_{close} = 0, N_{oil} = 0, N_{p_clean} = 0, \neg H_p, N_{p_store} > 0, d_{plate} = 0\} \mapsto \{H_p\} \\
& \{N_{close} = 0, N_{oil} = 0, N_{p_clean} = 0, H_p, N_{p_store} > 0, d_{c1} > 0\} \mapsto \{d_{c1} \downarrow\} \\
& \{N_{close} = 0, N_{oil} = 0, N_{p_clean} = 0, H_p, N_{p_store} > 0, d_{c1} = 0\} \mapsto \{N_{p_store} \downarrow, \neg H_p\}
\end{aligned}$$

904 *Blender to countertop:*

$$\begin{aligned}
& \{N_b > 0, \neg H_b, d_{blender} > 0\} \mapsto \{d_{blender} \downarrow\} \\
& \{N_b > 0, \neg H_b, d_{blender} = 0\} \mapsto \{H_b\} \\
& \{H_b, N_b > 0, d_{ct} > 0\} \mapsto \{d_{ct} \downarrow\} \\
& \{H_b, N_b > 0, d_{ct} = 0\} \mapsto \{N_b \downarrow, \neg H_b\}
\end{aligned}$$

905 *Apple to fridge:*

$$\begin{aligned}
& \{N_{close} = 0, \neg H_f, d_{apple} > 0\} \mapsto \{d_{apple} \downarrow\} \\
& \{N_{close} = 0, \neg H_f, d_{apple} = 0\} \mapsto \{H_f\} \\
& \{N_{close} = 0, H_f, N_a > 0, d_{fr} > 0\} \mapsto \{d_{fr} \downarrow\} \\
& \{N_{close} = 0, H_f, N_a > 0, d_{fr} = 0\} \mapsto \{N_a \downarrow, \neg H_f\}
\end{aligned}$$

906 *Casserole to fridge:*

$$\begin{aligned}
& \{N_{close} = 0, \neg H_f, d_{casserole} > 0\} \mapsto \{d_{casserole} \downarrow\} \\
& \{N_{close} = 0, \neg H_f, d_{casserole} = 0\} \mapsto \{H_f\} \\
& \{N_{close} = 0, H_f, N_{cas} > 0, d_{fr} > 0\} \mapsto \{d_{fr} \downarrow\} \\
& \{N_{close} = 0, H_f, N_{cas} > 0, d_{fr} = 0\} \mapsto \{N_{cas} \downarrow, \neg H_f\}
\end{aligned}$$

907 *Cabinet dusting:*

$$\begin{aligned}
& \{\neg H_r, d_{rag} > 0\} \mapsto \{d_{rag} \downarrow\} && ; \text{get rag} \\
& \{\neg H_r, d_{rag} = 0\} \mapsto \{H_r\} \\
& \{H_r, N_{cab} > 0, d_c > 0\} \mapsto \{d_c \downarrow\} && ; \text{clean cabinet} \\
& \{H_r, N_{cab} > 0, d_c = 0\} \mapsto \{N_{cab} \downarrow, \neg H_r\}
\end{aligned}$$

908 *Rag near/inside sink:*

$$\begin{aligned}
& \{N_{cab} = 0, N_{p_clean} = 0, \neg H_r, d_{rag} > 0\} \mapsto \{d_{rag} \downarrow\} \\
& \{N_{cab} = 0, N_{p_clean} = 0, \neg H_r, d_{rag} = 0\} \mapsto \{H_r\} \\
& \{N_{cab} = 0, N_{p_clean} = 0, H_r, N_r > 0, d_{si} > 0\} \mapsto \{d_{si} \downarrow\} \\
& \{N_{cab} = 0, N_{p_clean} = 0, H_r, N_r > 0, d_{si} = 0\} \mapsto \{N_r \downarrow, \neg H_r\}
\end{aligned}$$

909 *Soap next to sink:*

$$\begin{aligned}
& \{N_{cab} = 0, N_{p_clean} = 0, \neg H_s, d_{soap} > 0\} \mapsto \{d_{soap} \downarrow\} \\
& \{N_{cab} = 0, N_{p_clean} = 0, \neg H_s, d_{soap} = 0\} \mapsto \{H_s\} \\
& \{N_{cab} = 0, N_{p_clean} = 0, H_s, N_s > 0, d_{si} > 0\} \mapsto \{d_{si} \downarrow\} \\
& \{N_{cab} = 0, N_{p_clean} = 0, H_s, N_s > 0, d_{si} = 0\} \mapsto \{N_s \downarrow, \neg H_s\}
\end{aligned}$$

E.4 Task 4: Organizing File Cabinet

Features:

- H_m : holding a marker
- H_f : holding a folder
- H_d : holding a document
- p_m : distance to the nearest marker
- p_f : distance to the nearest folder
- p_d : distance to the nearest document
- p_t : distance to a table
- p_c : distance to a valid cabinet target
- N_m : number of markers not on a table
- N_f : number of folders not in a cabinet
- N_d : number of documents not in a cabinet
- N_o : number of closed cabinets

Width-0 Sketch:

Opening Cabinet:

$\{N_o > 0, p_c > 0\} \mapsto \{p_c \downarrow\}$; move toward a closed cabinet
 $\{N_o > 0, p_c = 0\} \mapsto \{N_o \downarrow\}$; open the cabinet when reachable

Marker task:

$\{N_m > 0, \neg H_m, p_m > 0\} \mapsto \{p_m \downarrow, p_t?\}$; move toward the nearest marker
 $\{N_m > 0, \neg H_m, p_m = 0\} \mapsto \{H_m, p_t?\}$; pick up the marker when reachable
 $\{N_m > 0, H_m, p_t > 0\} \mapsto \{p_t \downarrow\}$; move toward a table
 $\{N_m > 0, H_m, p_t = 0\} \mapsto \{N_m \downarrow, \neg H_m, p_m?\}$; put marker on table

Folder task:

$\{N_f > 0, N_o = 0, \neg H_f, p_f > 0\} \mapsto \{p_f \downarrow, p_c?\}$
 $\{N_f > 0, N_o = 0, \neg H_f, p_f = 0\} \mapsto \{H_f, p_c?\}$
 $\{N_f > 0, N_o = 0, H_f, p_c > 0\} \mapsto \{p_c \downarrow\}$
 $\{N_f > 0, N_o = 0, H_f, p_c = 0\} \mapsto \{N_f \downarrow, \neg H_f, p_f?\}$

Document task:

$\{N_d > 0, N_o = 0, \neg H_d, p_d > 0\} \mapsto \{p_d \downarrow, p_c?\}$
 $\{N_d > 0, N_o = 0, \neg H_d, p_d = 0\} \mapsto \{H_d, p_c?\}$
 $\{N_d > 0, N_o = 0, H_d, p_c > 0\} \mapsto \{p_c \downarrow\}$
 $\{N_d > 0, N_o = 0, H_d, p_c = 0\} \mapsto \{N_d \downarrow, \neg H_d, p_d?\}$

E.5 Task 5: Thawing Frozen Food

Features:

- H_{fish} : holding a fish
- H_{olive} : holding an olive
- H_{date} : holding a date
- p_{fish} : distance to the nearest fish
- p_{olive} : distance to the nearest olive
- p_{date} : distance to the nearest date
- p_{sink} : distance to a sink
- N_{fish} : number of fish not next to a sink
- N_{olive} : number of olives not next to a sink
- N_{date} : number of dates not next to a fish

Width-0 Sketch:

Fish Task:

$\{\neg H_{fish}, p_{fish} > 0\} \mapsto \{p_{fish} \downarrow\}$
 $\{\neg H_{fish}, p_{fish} = 0\} \mapsto \{H_{fish}\}$
 $\{H_{fish}, p_{sink} > 0\} \mapsto \{p_{sink} \downarrow\}$
 $\{H_{fish}, p_{sink} = 0\} \mapsto \{N_{fish} \downarrow, \neg H_{fish}, p_{fish}?\}$

943 *Olive Task:*

$$\begin{aligned}\{\neg H_{olive}, p_{olive} > 0\} &\mapsto \{p_{olive} \downarrow\} \\ \{\neg H_{olive}, p_{olive} = 0\} &\mapsto \{H_{olive}\} \\ \{H_{olive}, p_{sink} > 0\} &\mapsto \{p_{sink} \downarrow\} \\ \{H_{olive}, p_{sink} = 0\} &\mapsto \{N_{olive} \downarrow, \neg H_{olive}, p_{olive}?\}\end{aligned}$$

944 *Date Task:*

$$\begin{aligned}\{\neg H_{date}, N_{fish\ olive} = 0, p_{date} > 0\} &\mapsto \{p_{date} \downarrow\} \\ \{\neg H_{date}, N_{fish\ olive} = 0, p_{date} = 0\} &\mapsto \{H_{date}\} \\ \{H_{date}, N_{fish\ olive} = 0, p_{fish} > 0\} &\mapsto \{p_{fish} \downarrow\} \\ \{H_{date}, N_{fish\ olive} = 0, p_{fish} = 0\} &\mapsto \{N_{date} \downarrow, \neg H_{date}, p_{date}?\}\end{aligned}$$

945 **E.6 Task 6: Making Tea**

946 **Features:**

- 947 • H_l : holding a lemon
- 948 • H_{knife} : holding a knife
- 949 • H_t : holding a teapot
- 950 • H_{tb} : holding a tea bag
- 951 • p_l : distance to the nearest lemon
- 952 • p_{knife} : distance to the knife
- 953 • p_t : distance to the nearest teapot
- 954 • p_{tb} : distance to the nearest tea bag
- 955 • p_s : distance to the stove
- 956 • p_{sink} : distance to the sink
- 957 • $is_sliced(lemon)$: whether the lemon is sliced
- 958 • $is_toggled(stove)$: whether the stove is toggled on
- 959 • $is_toggled(sink)$: whether the sink is toggled on
- 960 • $is_soaked(tea_bag)$: whether the tea bag is soaked
- 961 • $atsamelocation(tea_bag, teapot)$: tea bag at same location as teapot
- 962 • $onTop(teapot, stove)$: teapot on stove

963 **Width-0 Sketch:**

964 *Toggle stove:*

$$\begin{aligned}\{\neg is_toggled(stove), p_s > 0\} &\mapsto \{p_s \downarrow\} \\ \{\neg is_toggled(stove), p_s = 0\} &\mapsto \{is_toggled(stove)\}\end{aligned}$$

965 *Slice lemon:*

$$\begin{aligned}\{\neg H_{knife}, p_{knife} > 0, \neg is_sliced(lemon)\} &\mapsto \{p_{knife} \downarrow\} \\ \{\neg H_{knife}, p_{knife} = 0, \neg is_sliced(lemon)\} &\mapsto \{H_{knife}\} \\ \{H_{knife}, \neg is_sliced(lemon), p_l > 0\} &\mapsto \{p_l \downarrow\} \\ \{H_{knife}, \neg is_sliced(lemon), p_l = 0\} &\mapsto \{is_sliced(lemon)\} \\ \{H_{knife}, is_sliced(lemon)\} &\mapsto \{\neg H_{knife}, p_{knife}?\}\end{aligned}$$

966 *Put teapot on stove and put tea bag in teapot:*

$$\begin{aligned}\{\neg atsameloc(tb, tp), \neg onTop(tp, st), \neg H_t, p_t > 0\} &\mapsto \{p_t \downarrow\} \\ \{\neg atsameloc(tb, tp), \neg onTop(tp, st), \neg H_t, p_t = 0\} &\mapsto \{H_t\} \\ \{\neg atsameloc(tb, tp), \neg onTop(tp, st), H_t, p_s > 0\} &\mapsto \{p_s \downarrow\} \\ \{\neg atsameloc(tb, tp), \neg onTop(tp, st), H_t, p_s = 0\} &\mapsto \{onTop(tp, st), \neg H_t, p_t?\} \\ \{\neg atsameloc(tb, tp), onTop(tp, st), \neg H_{tb}, p_{tb} > 0\} &\mapsto \{p_{tb} \downarrow\} \\ \{\neg atsameloc(tb, tp), onTop(tp, st), \neg H_{tb}, p_{tb} = 0\} &\mapsto \{H_{tb}\} \\ \{\neg atsameloc(tb, tp), onTop(tp, st), H_{tb}, p_t > 0\} &\mapsto \{p_t \downarrow\} \\ \{H_{tb}, \neg atsameloc(tb, tp), onTop(tp, st), p_t = 0\} &\mapsto \{atsameloc(tb, tp), \neg H_{tb}, p_{tb}?\}\end{aligned}$$

E.7 Task 7: Opening Packages

Features:

- p : distance to the nearest package
- N : number of unopened packages

Width-0 Sketch:

$$\begin{aligned}\{N > 0, p > 0\} &\mapsto \{p \downarrow\} && ; \text{ move toward the nearest unopened package} \\ \{N > 0, p = 0\} &\mapsto \{N \downarrow\} && ; \text{ open the package when reachable}\end{aligned}$$

E.8 Task 8: Boxing Books Up for Storage

Features:

- H : holding an unboxed book
- p : distance to the nearest unboxed book
- b : distance to a valid box target
- N : number of unboxed books

Width-0 Sketch:

$$\begin{aligned}\{\neg H, p > 0\} &\mapsto \{p \downarrow, b?\} && ; \text{ move toward the nearest unboxed book} \\ \{\neg H, p = 0\} &\mapsto \{H\} && ; \text{ pick up the book when reachable} \\ \{H, N > 0, b > 0\} &\mapsto \{b \downarrow\} && ; \text{ move toward a chosen box spot} \\ \{H, N > 0, b = 0\} &\mapsto \{\neg H, N \downarrow, p?\} && ; \text{ place the book into the box}\end{aligned}$$

E.9 Task 9: Collect Misplaced Items

Features:

- H_s : holding a gym shoe
- H_n : holding a necklace
- H_b : holding a notebook
- H_k : holding a sock
- p_s : distance to the nearest gym shoe
- p_n : distance to the nearest necklace
- p_b : distance to the nearest notebook
- p_k : distance to the nearest sock
- p_t : distance to a table
- N_s : number of gym shoes not on a table
- N_n : number of necklaces not on a table
- N_b : number of notebooks not on a table
- N_k : number of socks not on a table

Width-0 Sketch:

Gym shoe task:

$$\begin{aligned}\{\neg H_s, N_s > 0, p_s > 0\} &\mapsto \{p_s \downarrow, p_t?\} \\ \{\neg H_s, N_s > 0, p_s = 0\} &\mapsto \{H_s, p_t?\} \\ \{H_s, N_s > 0, p_t > 0\} &\mapsto \{p_t \downarrow\} \\ \{H_s, N_s > 0, p_t = 0\} &\mapsto \{N_s \downarrow, \neg H_s, p_s?\}\end{aligned}$$

Necklace task:

$$\begin{aligned}\{\neg H_n, N_n > 0, p_n > 0\} &\mapsto \{p_n \downarrow, p_t?\} \\ \{\neg H_n, N_n > 0, p_n = 0\} &\mapsto \{H_n, p_t?\} \\ \{H_n, N_n > 0, p_t > 0\} &\mapsto \{p_t \downarrow\} \\ \{H_n, N_n > 0, p_t = 0\} &\mapsto \{N_n \downarrow, \neg H_n, p_n?\}\end{aligned}$$

Notebook task:

$$\begin{aligned}\{\neg H_b, N_b > 0, p_b > 0\} &\mapsto \{p_b \downarrow, p_t?\} \\ \{\neg H_b, N_b > 0, p_b = 0\} &\mapsto \{H_b, p_t?\} \\ \{H_b, N_b > 0, p_t > 0\} &\mapsto \{p_t \downarrow\} \\ \{H_b, N_b > 0, p_t = 0\} &\mapsto \{N_b \downarrow, \neg H_b, p_b?\}\end{aligned}$$

998 *Sock task:*

$$\begin{aligned}\{\neg H_k, N_k > 0, p_k > 0\} &\mapsto \{p_k \downarrow, p_t?\} \\ \{\neg H_k, N_k > 0, p_k = 0\} &\mapsto \{H_k, p_t?\} \\ \{H_k, N_k > 0, p_t > 0\} &\mapsto \{p_t \downarrow\} \\ \{H_k, N_k > 0, p_t = 0\} &\mapsto \{N_k \downarrow, \neg H_k, p_k?\}\end{aligned}$$

999 **E.10 Task 10: Putting Away Dishes After Cleaning**

1000 **Features:**

- 1001 • H : holding a plate
- 1002 • p : distance to the nearest plate
- 1003 • b : distance to a valid cabinet target
- 1004 • N : number of unboxed plates
- 1005 • is_opened : whether the cabinet is opened

1006 **Width-0 Sketch:**

$$\begin{aligned}\{\neg \text{is_opened}, b > 0\} &\mapsto \{b \downarrow\} && ; \text{ move toward the cabinet} \\ \{\neg \text{is_opened}, b = 0\} &\mapsto \{\text{is_opened}\} && ; \text{ open the cabinet when reachable} \\ \{\neg H, N > 0, p > 0, \text{is_opened}\} &\mapsto \{p \downarrow, b?\} \\ \{\neg H, N > 0, p = 0, \text{is_opened}\} &\mapsto \{H, b?\} \\ \{H, N > 0, b > 0, \text{is_opened}\} &\mapsto \{b \downarrow\} \\ \{H, N > 0, b = 0, \text{is_opened}\} &\mapsto \{\neg H, N \downarrow, p?\}\end{aligned}$$

1007 **E.11 Task 11: Washing Pots and Pans**

1008 **Features:**

- 1009 • H_r : holding a scrub_brush
- 1010 • H_f : holding a focused item (teapot, kettle, or pan)
- 1011 • p_r : distance to the nearest scrub_brush
- 1012 • p_f : distance to the nearest focused item
- 1013 • p_s : distance to the sink
- 1014 • p_c : distance to a valid cabinet target
- 1015 • Clean_f : whether the focused item is clean (not stained)
- 1016 • N_f : number of focused items not stored
- 1017 • is_toggled : whether the sink is toggled on
- 1018 • is_soaked : whether the scrub_brush is soaked
- 1019 • is_opened : whether the cabinet is opened

1020 **Width-0 Sketch:**

1021 *Clean task:*

$$\begin{aligned}\{\neg H_r, N_f > 0, p_r > 0\} &\mapsto \{p_r \downarrow, p_f?, p_s?\} \\ \{\neg H_r, N_f > 0, p_r = 0\} &\mapsto \{H_r, p_f?, p_s?\} \\ \{\neg \text{is_toggled}, p_s > 0\} &\mapsto \{p_s \downarrow, p_f?, p_r?\} \\ \{\neg \text{is_toggled}, p_s = 0\} &\mapsto \{\text{is_toggled}, p_f?, p_r?\} \\ \{H_r, \text{is_toggled}, \neg \text{is_soaked}, p_s = 0\} &\mapsto \{\text{is_soaked}, \neg H_r, p_f?, p_r = 0\} \\ \{\neg H_r, \text{is_soaked}, \neg \text{Clean}_f, p_r = 0\} &\mapsto \{H_r, p_f?\} \\ \{H_r, \text{is_soaked}, \neg \text{Clean}_f, p_f > 0\} &\mapsto \{p_f \downarrow, p_r?, p_s?\} \\ \{H_r, \text{is_soaked}, \neg \text{Clean}_f, p_f = 0\} &\mapsto \{\text{Clean}_f, \neg H_r, p_r?, p_s?\}\end{aligned}$$

1022 *Store task:*

$$\begin{aligned}\{\neg \text{is_opened}, p_c > 0\} &\mapsto \{p_c \downarrow\} \\ \{\neg \text{is_opened}, p_c = 0\} &\mapsto \{\text{is_opened}\} \\ \{\neg H_f, N_f > 0, \text{Clean}_f, p_f > 0\} &\mapsto \{p_f \downarrow, p_c?\} \\ \{\neg H_f, N_f > 0, \text{Clean}_f, p_f = 0\} &\mapsto \{H_f, p_c?\} \\ \{H_f, N_f > 0, \text{Clean}_f, p_c > 0\} &\mapsto \{p_c \downarrow\} \\ \{H_f, N_f > 0, \text{Clean}_f, p_c = 0\} &\mapsto \{\neg H_f, N_f \downarrow, p_f?, p_r?, p_s?\}\end{aligned}$$

E.12 Task 12: Cleaning Shoes

Features:

- H_r : holding a rag
- p_r : distance to the nearest rag
- p_s : distance to the sink
- p_f : distance to the nearest focused shoe
- N_f : number of focused shoes not cleaned
- is_toggled : whether the sink is toggled on
- is_soaked : whether the rag is soaked
- onfloor : whether the rag is on the floor

Width-0 Sketch:

Pick up rag and soak it:

$$\begin{aligned}
 \{\neg H_r, N_f > 0, p_r > 0\} &\mapsto \{p_r \downarrow, p_f?, p_s?\} \\
 \{\neg H_r, N_f > 0, p_r = 0\} &\mapsto \{H_r, p_f?, p_s?\} \\
 \{H_r, \neg \text{is_toggled}, p_s > 0\} &\mapsto \{p_s \downarrow, p_f?, p_r?\} \\
 \{H_r, \neg \text{is_toggled}, p_s = 0\} &\mapsto \{\text{is_toggled}, p_f?, p_r?\} \\
 \{H_r, \text{is_toggled}, \neg \text{is_soaked}, p_s > 0\} &\mapsto \{p_s \downarrow\} \\
 \{H_r, \text{is_toggled}, \neg \text{is_soaked}, p_s = 0\} &\mapsto \{\text{is_soaked}, \neg H_r, p_f?, p_r = 0\}
 \end{aligned}$$

Clean task:

$$\begin{aligned}
 \{\neg H_r, \text{is_soaked}, N_f > 0, p_r > 0\} &\mapsto \{p_r \downarrow\} \\
 \{\neg H_r, \text{is_soaked}, N_f > 0, p_r = 0\} &\mapsto \{H_r, p_f?\} \\
 \{H_r, \text{is_soaked}, N_f > 0, p_f > 0\} &\mapsto \{p_f \downarrow, p_r?, p_s?\} \\
 \{H_r, \text{is_soaked}, N_f > 0, p_f = 0\} &\mapsto \{N_f \downarrow, \neg H_r, p_r?, p_s?\}
 \end{aligned}$$

Put towel on floor at the end:

$$\{N_f = 0, \neg \text{onfloor}, H_r\} \mapsto \{\neg H_r, \text{onfloor}\}$$

E.13 Task 13: Installing a Printer

Features:

- H : holding a printer
- p : distance to the nearest printer
- b : distance to a table
- is_toggled : whether the printer is toggled on
- N : number of uninstalled printers

Width-0 Sketch:

$$\begin{aligned}
 \{\neg H, N > 0, p > 0\} &\mapsto \{p \downarrow, b?\} \\
 \{\neg H, \neg \text{is_toggled}, N > 0, p = 0\} &\mapsto \{\neg H, \text{is_toggled}\} \\
 \{\neg H, \text{is_toggled}, N > 0, p = 0\} &\mapsto \{H, b?\} \\
 \{H, N > 0, b > 0\} &\mapsto \{b \downarrow\} \\
 \{H, N > 0, b = 0\} &\mapsto \{\neg H, N \downarrow, p = 0\}
 \end{aligned}$$

E.14 Task 14: Watering Houseplants

Features:

- H : holding a pot plant
- p : distance to the nearest pot plant
- b : distance to the sink
- is_toggled : whether the sink is toggled on
- N : number of unsoaked pot plants

1052 **Width-0 Sketch:**

$$\begin{aligned}
\{\neg H, N > 0, p > 0\} &\mapsto \{p \downarrow, b?\} \\
\{\neg H, N > 0, p = 0\} &\mapsto \{H, b?\} \\
\{\neg \text{is_toggled}, b > 0\} &\mapsto \{b \downarrow, p?\} \\
\{\neg \text{is_toggled}, b = 0\} &\mapsto \{\text{is_toggled}, p?\} \\
\{H, N > 0, \text{is_toggled}, b > 0\} &\mapsto \{b \downarrow\} \\
\{H, N > 0, \text{is_toggled}, b = 0\} &\mapsto \{\neg H, N \downarrow, p = 0\}
\end{aligned}$$

1053 **E.15 Task 15: Cleaning a Car**

1054 **Features:**

- 1055 • H_r : holding a rag
- 1056 • H_s : holding a soap
- 1057 • p_r : distance to the nearest rag
- 1058 • p_s : distance to the nearest soap
- 1059 • p_c : distance to the nearest focused car
- 1060 • p_b : distance to the bucket
- 1061 • N_c : number of focused cars not cleaned
- 1062 • N_{in_bucket} : number of items (rag and soap) not in the bucket
- 1063 • inside_rag : whether the rag is in the bucket

1064 **Width-0 Sketch:**

1065 *Clean task:*

$$\begin{aligned}
\{\neg H_r, N_c > 0, p_r > 0\} &\mapsto \{p_r \downarrow, p_c?\} \\
\{\neg H_r, N_c > 0, p_r = 0\} &\mapsto \{H_r, p_c?\} \\
\{H_r, N_c > 0, p_c > 0\} &\mapsto \{p_c \downarrow, p_r?\} \\
\{H_r, N_c > 0, p_c = 0\} &\mapsto \{N_c \downarrow, \neg H_r, p_r?\}
\end{aligned}$$

1066 *Put rag in the bucket:*

$$\begin{aligned}
\{\neg \text{inside_rag}, \neg H_r, N_c = 0, N_{in_bucket} > 0, p_r > 0\} &\mapsto \{p_r \downarrow, p_b?\} \\
\{\neg \text{inside_rag}, \neg H_r, N_c = 0, N_{in_bucket} > 0, p_r = 0\} &\mapsto \{H_r, p_b?\} \\
\{H_r, \neg \text{inside_rag}, N_c = 0, N_{in_bucket} > 0, p_b > 0\} &\mapsto \{p_b \downarrow, p_r?\} \\
\{H_r, \neg \text{inside_rag}, N_c = 0, N_{in_bucket} > 0, p_b = 0\} &\mapsto \{\text{inside_rag}, \neg H_r, N_{in_bucket} \downarrow, p_r?\}
\end{aligned}$$

1067 *Put soap in the bucket:*

$$\begin{aligned}
\{\neg \text{inside_rag}, \neg H_s, N_c = 0, N_{in_bucket} > 0, p_s > 0\} &\mapsto \{p_s \downarrow, p_b?\} \\
\{\neg \text{inside_rag}, \neg H_s, N_c = 0, N_{in_bucket} > 0, p_s = 0\} &\mapsto \{H_s, p_b?\} \\
\{H_s, \text{inside_rag}, N_c = 0, N_{in_bucket} > 0, p_b > 0\} &\mapsto \{p_b \downarrow, p_s?\} \\
\{H_s, \text{inside_rag}, N_c = 0, N_{in_bucket} > 0, p_b = 0\} &\mapsto \{\neg H_s, N_{in_bucket} \downarrow, p_r?, p_c?\}
\end{aligned}$$

1068 **E.16 Task 16: Storing Food**

1069 **Features:**

- 1070 • H : holding an unstored cookable item (food)
- 1071 • p : distance to the nearest unstored cookable item
- 1072 • s : distance to a cabinet
- 1073 • is_opened : whether the cabinet is opened
- 1074 • N_f : number of unstored cookable items

1075 **Width-0 Sketch:**

$$\begin{aligned}
\{\neg H, N_f > 0, p > 0\} &\mapsto \{p \downarrow, s?\} \\
\{\neg H, N_f > 0, p = 0\} &\mapsto \{H, s?\} \\
\{H, N_f > 0, s > 0\} &\mapsto \{s \downarrow, p?\} \\
\{\neg \text{is_opened}, s = 0\} &\mapsto \{\text{is_opened}, p?\} \\
\{H, N_f > 0, \text{is_opened}, s = 0\} &\mapsto \{\neg H, N_f \downarrow, p = 0\}
\end{aligned}$$

E.17 Task 17: Throwing Away Leftovers

1076

Features:

1077

- H : holding an unthrown hamburger
- p : distance to the nearest unthrown hamburger
- s : distance to an ashcan
- N : number of unthrown hamburgers

1078

1079

1080

1081

Width-0 Sketch:

1082

$$\begin{aligned}\{\neg H, N > 0, p > 0\} &\mapsto \{p \downarrow, s?\} \\ \{\neg H, N > 0, p = 0\} &\mapsto \{H, s?\} \\ \{H, N > 0, s > 0\} &\mapsto \{s \downarrow\} \\ \{H, N > 0, s = 0\} &\mapsto \{\neg H, N \downarrow, p = 0\}\end{aligned}$$

E.18 Task 18: Moving Boxes to Storage

1083

Features:

1084

- H : holding an unstored box (carton)
- p : distance to the nearest unstored box
- s : distance to a valid storage target (shelf/location/dimension)
- n : number of unstored boxes

1085

1086

1087

1088

Width-0 Sketch:

1089

$$\begin{aligned}\{\neg H, p > 0\} &\mapsto \{p \downarrow, s?\} \\ \{\neg H, p = 0\} &\mapsto \{H\} \\ \{H, s > 0\} &\mapsto \{s \downarrow\} \\ \{H, n > 0, s = 0\} &\mapsto \{H?, n \downarrow, p?\}\end{aligned}$$

E.19 Task 19: Sorting Books

1090

Features:

1091

- H : holding an unsorted book (book or hardback)
- p : distance to the nearest unsorted book
- s : distance to a shelf
- N : number of unsorted books

1092

1093

1094

1095

Width-0 Sketch:

1096

$$\begin{aligned}\{\neg H, N > 0, p > 0\} &\mapsto \{p \downarrow, s?\} \\ \{\neg H, N > 0, p = 0\} &\mapsto \{H, s?\} \\ \{H, N > 0, s > 0\} &\mapsto \{s \downarrow\} \\ \{H, N > 0, s = 0\} &\mapsto \{\neg H, N \downarrow, p = 0\}\end{aligned}$$

E.20 Task 20: Setting Up Candles

1097

Features:

1098

- H : holding a candle
- p : distance to the nearest candle
- b : distance to a table
- N : number of unplaced candles
- N_t : number of unsettled tables (tables with less than 3 candles)

1099

1100

1101

1102

1103

Width-0 Sketch:

1104

$$\begin{aligned}\{\neg H, N > 0, N_t > 0, p > 0\} &\mapsto \{p \downarrow, b?\} \\ \{\neg H, N > 0, N_t > 0, p = 0\} &\mapsto \{H, b?\} \\ \{H, N > 0, N_t > 0, b > 0\} &\mapsto \{b \downarrow\} \\ \{H, N > 0, N_t > 0, b = 0\} &\mapsto \{\neg H, N \downarrow, N_t?, p = 0\}\end{aligned}$$

Task	Split	Max Width	Relative Granularity Bins		
			Fine	Medium	Coarse
opening_packages	ID	1	0	0	1
installing_a_printer	ID	2	0	1	2
laying_wood_floors	ID	2	0	1	2
moving_boxes_to_storage	ID	2	0	1	2
cleaning_a_car	ID	3	0	1	2,3
throwing_away_leftovers	ID	2	0	1	2
organizing_file_cabinet	ID	3	0	1	2,3
making_tea	ID	3	0	1	2,3
sorting_books	ID	2	0	1	2
collect_misplaced_items	ID	2	0	1	2
thawing_frozen_food	ID	2	0	1	2
boxing_books_up_for_storage	ID	2	0	1	2
storing_food	ID	3	0	1	2,3
preparing_salad	ID	4	0	1,2	3,4
putting_away_dishes_after_cleaning	ID	3	0	1	2,3
cleaning_up_the_kitchen_only	ID	5	0,1	2,3	4,5
setting_up_candles	ID	2	0	1	2
cleaning_shoes	OOD	3	0	1	2,3
watering_houseplants	OOD	3	0	1	2,3
washing_pots_and_pans	OOD	4	0	1,2	3,4

Table 7: Per-task instruction width ranges and relative granularity binning in MINI-BEHAVIOR-GRAN. Symbolic widths are normalized into Fine (Low), Medium, and Coarse (High) levels using a task-specific quantile-based strategy.

E.21 Instruction Generation Statistics

We used a template-based generator to translate the symbolic rule sets into natural language instructions, ensuring alignment with the VLA’s training data format. Table 8 presents the summary statistics for the generated instructions across the three granularity levels.

Table 8: Statistics of generated natural language instructions across granularity levels.

Granularity	Count	Avg Length (words)	Std Length	Min	Max	Unique Tokens	TTR (Type-Token Ratio)	Avg Sent. Length
Fine (F)	1636	41.12	11.36	19	87	140	0.0021	41.12
Medium (M)	859	38.62	12.76	22	81	108	0.0033	38.62
Coarse (C)	587	33.60	13.39	22	95	94	0.0048	33.60

E.22 Sample Generated Instructions by Granularity

To illustrate the qualitative differences between granularity levels, we provide five random samples for each category below. Note how Low (L) granularity explicitly specifies navigation and preconditions (e.g., “move toward,” “not yet at”), while High (H) granularity focuses almost exclusively on the final state change (e.g., “reduce the count of...”).

Fine Granularity (F) Samples

1. At step 11: When the count of uncleaned plate is above 0, not yet at sink 0 and sink is off, please try to move toward sink 0.
2. At step 8: When not yet at soap 0, not holding soap 0, the count of unwiped car is 0 and the count of rag not inside bucket is 0, please try to move toward soap 0.
3. At step 27: When the count of chip and oatmeal sugar and vegetable oil not inside cabinet is above 0, you are at oatmeal 1 and not holding oatmeal 1, please pick up oatmeal 1.
4. At step 15: When the count of plate not inside cabinet is above 0, you are at plate 3, the count of closed cabinet is 0 and not holding plate 3, please pick up plate 3.
5. At step 13: When the count of fish and olive not near sink is above 0, not yet at olive 0 and not holding olive 0, please try to move toward olive 0.

1. At step 2: When the count of plate not inside cabinet is above 0, the count of closed cabinet is 0 and not holding plate 2, please pick up plate 2.
 2. At step 6: When the count of uncleaned plate is above 0, the count of dry rag is above 0 and holding rag 0, please reduce the count of dry rag.
 3. At step 9: When the count of plate not inside cabinet 0 is above 0, the count of closed cabinet is 0, the count of vegetable oil not inside cabinet 1 is 0, the count of uncleaned plate is 0 and not holding plate 0, please pick up plate 0.
 4. At step 13: When the count of book not inside box is above 0 and not holding book 2, please pick up book 2.
 5. At step 9: When the count of chip and oatmeal sugar and vegetable oil not inside cabinet is above 0, holding chip 1 and the count of closed cabinet is 0, please not holding chip 1, then reduce the count of chip and oatmeal sugar and vegetable oil not inside cabinet.

1. At step 2: When the count of hamburger not inside ashcan is above 0, please reduce the count of hamburger not inside ashcan.
 2. At step 5: When the count of plate not inside cabinet is above 0 and the count of closed cabinet is 0, please reduce the count of plate not inside cabinet.
 3. At step 3: When the count of teapot not on stove is above 0, please reduce the count of teapot not on stove.
 4. At step 9: When the count of plate not inside cabinet is above 0 and the count of closed cabinet is 0, please reduce the count of plate not inside cabinet.
 5. At step 8: When the count of chip and oatmeal sugar and vegetable oil not inside cabinet is above 0 and the count of closed cabinet is 0, please reduce the count of chip and oatmeal sugar and vegetable oil not inside cabinet.

F Implementation Details

F.1 Model Architecture

Our policy is built upon the OpenVLA architecture (Kim, Pertsch, et al., 2024). Specifically, we utilize the Prismatic-7B backbone (Karamcheti et al., 2024), which integrates a frozen Llama-2-7B language model with fused visual features. The visual encoder consists of a dual-stream setup combining SigLIP (for semantic understanding) and DINOv2 (for fine-grained spatial geometry) (Oquab et al., 2024; Zhai et al., 2023).

F.2 Training Configuration

We fine-tune the model using Low-Rank Adaptation (LoRA) on the language model backbone, keeping the visual encoders frozen. All training setting variants (Pure, Centroid, Axial) share the same underlying training hyperparameters to ensure fair comparison.

Hardware and Duration: Training was conducted on a single node equipped with 8× NVIDIA A100 GPUs. A standard training run for 100,000 steps took approximately 23 hours to complete.

Table 9 summarizes the core hyperparameters used across our experiments.

Table 9: Hyperparameters for OpenVLA fine-tuning on MINI-BEHAVIOR-GRAN.

Hyperparameter	Value
Base Model	Prismatic-7B (Llama-2)
Visual Encoders	SigLIP + DINOv2
Parameter Efficient Tuning	LoRA (Rank $r = 32$)
Compute Hardware	8× NVIDIA A100
Training Duration	~23 hours
Optimization Steps	100,000
Steps Before Decay	80,000
Learning Rate	2.5×10^{-4}
Batch Size (per GPU)	7 – 8

1132

G Additional Experimental Results

1133 This section provides additional detailed per-task performance breakdowns, and failure mode distributions.

G.1 Detailed Task Performance

1135 Figure 11 details the success rates across all 20 tasks in the benchmark. We see some tasks are too difficult to solve and thus
1136 low across all instruction granularity. And then we also see the U shape trends in most tasks, showing the same phenomenon
1137 we discussed in (Q1) that both very fine and very coarse instructions are easier for VLA to follow.

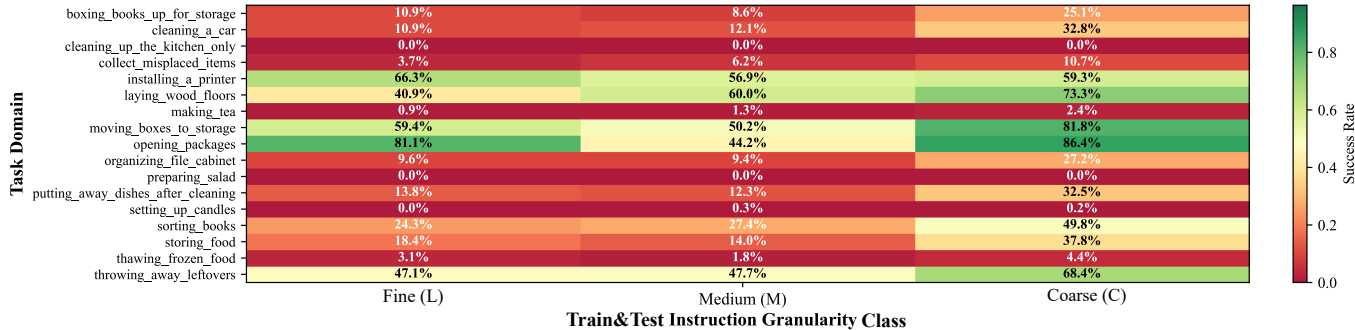


Figure 11: Detailed per-task success rates.

G.2 Failure Analysis

1139 Figure 12 presents the distribution of failure types across varying instruction widths. It confirms that low width encounters
1140 regression failures more frequently, while high width predominantly leads to stagnation failures. This aligns with our earlier
1141 observations in (Q3) regarding the distinct failure modes associated with different instruction granularities.

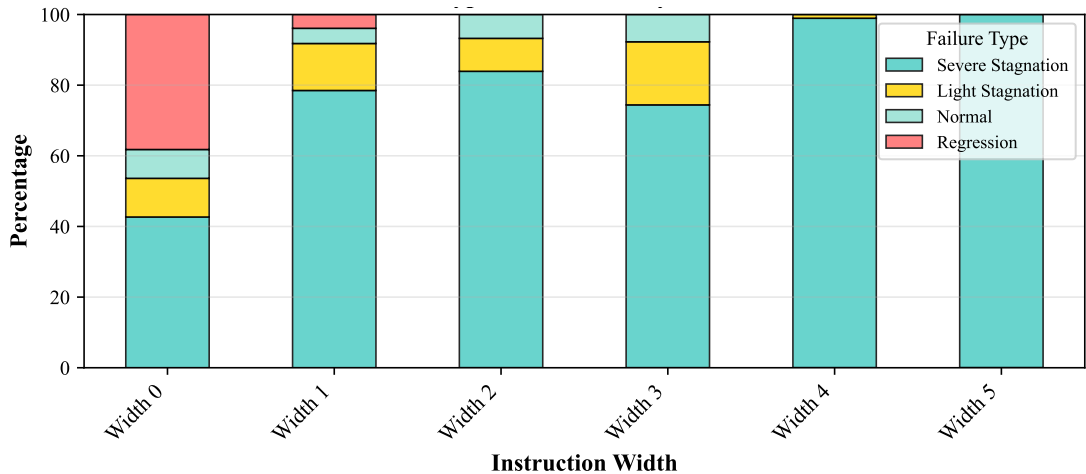


Figure 12: Failure type distribution across instruction widths.

References

1142 Hafner, Danijar (2021). “Benchmarking the Spectrum of Agent Capabilities”. In: *arXiv preprint arXiv:2109.06780*.
1143 Karamcheti, Siddharth et al. (2024). “Prismatic VLMs: Investigating the Design Space of Visually-Conditioned Language
1144 Models”. In: *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*.
1145 URL: <https://openreview.net/forum?id=6FXtu8clyp>.
1146 Li, Chengshu, Fei Xia, et al. (Aug. 2022). “iGibson 2.0: Object-Centric Simulation for Robot Learning of Everyday Household
1147 Tasks”. In: *Proceedings of the 5th Conference on Robot Learning*. Ed. by Aleksandra Faust, David Hsu, and Gerhard Neu-
1148 mann. Vol. 164. Proceedings of Machine Learning Research. PMLR, pp. 455–465. URL: <https://proceedings.mlr.press/v164/li22b.html>.
1149 Oquab, Maxime et al. (2024). “DINOv2: Learning Robust Visual Features without Supervision”. In: *Trans. Mach. Learn. Res.*
1150 2024.
1151 Zhai, Xiaohua et al. (2023). “Sigmoid Loss for Language Image Pre-Training”. In: *ICCV*. IEEE, pp. 11941–11952.